

LAASSO: Lattice-based anonymous single sign-on system with token hiding

Shunqi Ma^{a,b}, Rui Shi^c, Huiyan Chen^{id a,b,*}, Xi Lin^b, Huaxiong Wang^{id d}

^a School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing, China

^b Department of Cryptography Science and Technology, Beijing Electronics Science and Technology Institute, Beijing, China

^c School of Cyberspace Security, Hainan University, Haikou, Hainan, China

^d School of Physical and Mathematical Sciences, Nanyang Technological University, Nanyang, Singapore

ARTICLE INFO

Keywords:

Post-quantum security
Anonymous single sign-on
Attribute-based
Designated verifier
Token-hiding

ABSTRACT

The rapid digital transformation of the financial industry has amplified the demand for secure and privacy-preserving identity authentication across multiple services. Conventional single sign-on systems improve usability but expose sensitive client data to profiling and cross-service aggregation, creating severe privacy and compliance risks. Anonymous SSO (ASSO) schemes were introduced to mitigate these issues; however, existing constructions remain limited, as most rely on trusted third parties or centralized validators, lack fine-grained attribute disclosure, and depend on classical cryptographic assumptions that are vulnerable to quantum attacks.

In this work, we propose LAASSO, a lattice-based anonymous single sign-on system that achieves fine-grained attribute access control, designated-verifier authentication, and token hiding, while providing post-quantum security. LAASSO utilizes a lattice-based signature algorithm together with zero-knowledge proofs, enabling the client to selectively disclose attributes and to anonymously register and access services, while resisting collusion among verifiers. The client can hide an access token while specifying an authorized verifier. As a result, unauthorized verifiers will be unable to extract any information from the token. We formally define the system security model and evaluate its efficiency. Compared with existing schemes, LAASSO attains post-quantum security, thereby offering a practical and promising approach to privacy-preserving authentication in a post-quantum financial ecosystem.

1. Introduction

The rapid digital transformation of the financial industry has fundamentally reshaped service delivery for both individuals and enterprises. Modern financial institutions typically operate across diverse domains, including retail banking, wealth management for high-net-worth clients, enterprise-oriented supply chain finance, and open banking APIs for third-party platforms. While this multi-service ecosystem provides unprecedented convenience, it also poses significant challenges in identity management and privacy protection. In conventional settings, clients must repeatedly authenticate when accessing different services, which degrades the user experience and introduces significant communication overhead and operational costs. These limitations become particularly evident in multi-service environments where seamless, secure, and privacy-preserving access is crucial.

To alleviate this burden, single sign-on (SSO) systems [1,2] have been widely adopted. An SSO system enables a client to authenticate

once with a trusted identity server (IS) and then access multiple service providers (SPs) without requiring additional authentication. After completing the registration process, clients receive an access token for sign-on. This token contains some of the client's private information. While this approach enhances usability, it also centralizes identity management and amplifies privacy risks. Attackers can obtain clients' sensitive information by exploiting the data embedded in tokens, such as behavioral patterns, service preferences, and transaction histories. This information may be aggregated and exploited by service providers or third parties, resulting in unauthorized profiling, commercial misuse, or regulatory violations. These risks are particularly acute in the financial sector, which is subject to strict data protection laws, including the General Data Protection Regulation (GDPR) in the European Union and the Personal Information Protection Law (PIPL) in China. Both require that only the "minimum necessary" data be disclosed, and that personal and transactional data be accessed strictly within their intended scope.

* Corresponding author.

E-mail addresses: mashunqi@bupt.edu.cn (S. Ma), shir@hainanu.edu.cn (R. Shi), chenhy@besti.edu.cn (H. Chen), linxi18@mails.ucas.ac.cn (X. Lin), HXWang@ntu.edu.sg (H. Wang).

<https://doi.org/10.1016/j.jisa.2026.104590>

Available online 31 July 2026

2214-2126/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

To address these privacy concerns, anonymous SSO (ASSO) systems have been proposed.¹ Unlike conventional SSO, ASSO systems decouple authentication from real-world identities, allowing clients to obtain credentials after they complete identity verification. The client then uses a legal credential to access the service, with identity being concealed. Early ASSO constructions relied on trusted third parties (TTPs) to generate one-time pseudonyms [3], thereby achieving unlinkability of the client's identities; however, they may be vulnerable to single-point failure [4]. Such dependency also conflicts with modern distributed service ecosystems, where trust minimization is critical.

Later, to reduce reliance on TTPs, Wang et al. [5] presented an ASSO construction based on dynamic group signatures. Group signatures inherently provide signer anonymity. During registration, clients receive a signature on their public key from the TTP, which they subsequently combine with their private key to generate the registration credential. During service access, clients generate a group signature on a random message, accompanied by a zero-knowledge proof demonstrating possession of their private key. This enables SPs to perform offline verification without compromising client anonymity. While this group signature-based approach offers both client anonymity and traceability capabilities, a significant limitation emerges. All SPs retain the ability to verify any client's service request, which may create privacy vulnerabilities. Asghar et al. [6] introduced an alternative ASSO system based on credential-based authentication, which enables ISs to generate credentials that associate a client's public key with specific attribute sets. During service access, clients maintain the capability to selectively disclose only the necessary attributes to SPs, thereby preserving client privacy through controlled information sharing. However, despite providing stronger unlinkability properties, these constructions rely on an implicit assumption that all service providers operate honestly and refrain from collusion. This assumption does not hold in practice.

Privacy can be compromised not only through identity linking but also through service access linking. Han et al. [7] proposed a dynamic SSO construction by integrating broadcast encryption, strongly unforgeable signatures, and zero-knowledge proofs. In their framework, broadcast encryption ensures that only authorized service providers can recover the decryption key from the credential, while revoked or unauthorized providers learn nothing. Strong, unforgeable signatures prevent malicious users or service providers from forging credentials, and zero-knowledge proofs enable users to demonstrate possession of the private key corresponding to the public key in the credential, thereby preventing impersonation attacks. However, this approach does not achieve unlinkability of users, since all authorized service providers can learn the client's public key.

To mitigate the risk of verifier collusion, Han et al. [8] implemented the ASSO system based on BBS+ signatures [9] and zero-knowledge proofs [10], and first proposed the designated-verifier paradigm. Only the designated service provider can verify the client's service request. Clients receive tickets issued by the IS, enabling them to access services. Each ticket embeds multiple authentication tags, which are restricted to verification by the intended verifiers only. These verifiers can verify the correctness of their respective tags, but cannot associate them with the client's specific service request. However, the registration phase may lead to overexposure of client attributes and even leakage of ticket metadata, such as the validity period.

Proxy re-verification technology [11] enhances the system's availability by enabling a backup validator, but the stability of the verification service remains unimproved. Shi et al. [12], inspired by attribute-based ticketing systems [13–15], proposed PriSign, which integrates attribute-based anonymous credentials with token hiding based on functional encryption, achieving fine-grained attribute disclosure and service unlinkability. This design enables clients to prevent unauthorized

verifiers from obtaining service request information, providing stronger security guarantees for service unlinkability. The attribute-based anonymous credential scheme supports user attribute disclosure policies to obtain different types of credentials. Clients can hide access tokens according to verifier policies and specify authorized verifiers for the tokens. However, PriSign and similar schemes rely on bilinear pairings and other classical cryptographic assumptions, making them vulnerable to quantum adversaries.

1.1. Motivation

As previously discussed, although several ASSO constructions have been proposed, current schemes still encounter three challenges. First, most existing solutions lack comprehensive support for fine-grained attribute disclosure, which is crucial in financial services where sensitive attributes must be shared selectively. For example, a client accessing retail banking should disclose only purchase-related information, whereas wealth management services may require proof of age or account balance, but not a home address. Current schemes either enforce over-disclosure or fail to protect auxiliary metadata, exposing clients to privacy leakage. Second, many ASSO constructions assume honest verifiers or rely on central validators, leaving the system vulnerable to insider threats and operational failures. The 2024 Verizon Data Breach Investigations Report reported 3348 incidents in the financial and insurance industry, with 31% originating internally and 68% involving human factors, underscoring the systemic risks of centralized trust.² Third, and most critically, all existing ASSO systems are built upon classical hardness assumptions such as discrete logarithms or bilinear pairings [3,5–8,11,12], which are known to be vulnerable to polynomial-time attacks by quantum adversaries. This makes them unsuitable for long-term deployment in security-critical industries such as finance.

Existing approaches solve parts of the ASSO problem but fail to provide a comprehensive solution. TTP-based and group signature-based schemes provide unlinkability but lack fine-grained disclosure and token privacy. Designated-verifier and proxy re-verification systems address verifier collusion, but either leak metadata or rely on central authorities. Attribute-based credential systems like PriSign achieve selective disclosure and token hiding but remain vulnerable to quantum attacks. These limitations highlight the urgent need for a post-quantum secure ASSO framework that integrates fine-grained attribute-based access control, token hiding, and designated-verifier authentication.

Lattice-based schemes, such as group signatures [16–18], anonymous credentials [19–21], and zero-knowledge proofs [22–24], provide a promising foundation in this direction. However, research on lattice-based post-quantum ASSO systems remains limited. Hard problems such as Modular Learning With Errors (MLWE) and Modular Short Integer Solutions (MSIS) are widely conjectured to resist quantum attacks and have been standardized by NIST as the basis for post-quantum cryptography. Furthermore, lattice-based anonymous credentials naturally support selective attribute disclosure and efficient zero-knowledge proofs, making them highly suitable for building next-generation ASSO systems tailored to the stringent requirements of the financial industry.

1.2. Our contributions

This work presents a formal construction of a lattice-based anonymous attribute-based single sign-on (LAASSO) system. The research motivation originates from the financial industry's pressing demands for anonymity and post-quantum security, especially in critical financial environments, including banking core systems, securities trading platforms, insurance business management, and payment clearing systems, where users need both identity privacy protection and resistance against

¹ In ASSO systems, anonymity means that a user's identity is hidden, and multiple credentials issued to the same user are unlinkable to each other.

² <https://www.verizon.com/business/resources/reports>

future quantum computer threats to financial data security. Existing single sign-on systems generally lack quantum-resistant security, posing serious security risks in high-security financial contexts. Recent advances in lattice-based anonymous credential schemes [19–21,25] have provided crucial theoretical support for our LAASSO approach. Our specific contributions include:

- We present LAASSO, a concrete lattice-based anonymous single sign-on system that supports attribute-based access control, token hiding, and designated-verifier security in a unified post-quantum SSO framework. Our system involves three parties: clients, an identity server, and service providers. During the registration phase, clients can obtain registration credentials while revealing minimal information about their attributes to the IS. In the sign-on phase, clients anonymously access SPs by first obtaining tickets from the IS and then presenting zero-knowledge proofs of these tickets to access services. To prevent malicious service providers from misusing access tokens, we use public-key encryption to hide them, ensuring that only designated verifiers can verify and validate them.
- We instantiate and adapt the LAC (Lattice-based Anonymous Credentials) framework and the LNP zero-knowledge proof system [24] to realize the credential issuance, ticket obtaining, and anonymous sign-on procedures of LAASSO. We justify our choice of these cryptographic primitives as follows: the LAC system, based on NTRU signature schemes, offers smaller signature sizes and faster verification times than traditional lattice-based schemes. The LNP zero-knowledge proof system supports efficient proofs for linear relations with compact proof sizes, enabling authentication of client identities and attributes while preserving privacy.
- We provide formal security proofs for our proposed system, demonstrating that it achieves the desired security properties, including anonymity, unlinkability, and designated verifier security under lattice assumptions MSIS and MLWE. Additionally, we conduct a comprehensive performance analysis, evaluating the computational and communication overhead of our construction in comparison to the single sign-on system [26].

1.3. Organization

This work presents a comprehensive study of lattice-based anonymous single sign-on systems. Section 2 provides the necessary preliminaries. The LAASSO system model and security framework are formalized in Section 3. Section 4 demonstrates a practical instantiation of LAASSO. Section 5 evaluates its performance. Finally, Section 6 concludes the paper.

2. Preliminaries

2.1. Notation

We denote $[n] = \{1, 2, \dots, n\}$. Let $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^d + 1)$, where q is a prime and d is a power of 2. For any $\mathbf{x} \in \mathcal{R}_q$, $\tilde{\mathbf{x}} = \text{Coeffs}(\mathbf{x}) \in \mathbb{Z}^{md}$ is a function that extracts coefficients from $\mathbf{x}$. Conversely, $\mathbf{x} = \text{Coeffs}^{-1}(\tilde{\mathbf{x}}) \in \mathcal{R}_q$ is the inverse operation reconstructing m polynomials from its extracted coefficients. We denote

$$\text{rot}(a) = \begin{bmatrix} a_0 & -a_{d-1} & \cdots & -a_1 \\ a_1 & a_0 & \cdots & -a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_{d-1} & \cdots & a_1 & a_0 \end{bmatrix} \in \mathbb{Z}^{d \times d}$$

to be the rotation operator applied to coefficients of $a(X) = a_0 + a_1X + \cdots + a_{d-1}X^{d-1}$. For $\mathbf{A} = \{a_{i,j}\} \in \mathcal{R}_q^{m \times t}$, $\text{rot}(\mathbf{A}) = [\text{rot}(a_{i,j})] \in \mathbb{Z}^{md \times td}$. In this paper, $\{0, 1\}^*$ represents the set of all finite-length binary strings. For any $\psi \in \mathbb{N}$, define $\mathcal{S}_\psi = \{x \in \mathcal{R} : \|x\|_\infty \leq \psi\}$. Let $\rho_{\mathbf{s}, \mathbf{c}}(\mathbf{x})$ be an n -dimensional Gaussian function on $\mathbb{R}^n$. For any set $V \subset \mathbb{Z}^n$, $\rho_{\mathbf{s}, \mathbf{c}}(V) = \sum_{\mathbf{x} \in V} \rho_{\mathbf{s}, \mathbf{c}}(\mathbf{x})$. The discrete Gaussian distribution over V centered around $\mathbf{c} \in \mathbb{R}^n$ with

standard deviation $\mathbf{s}$ is defined as follows: for any $\mathbf{x} \in V$, $D_{V, \mathbf{s}, \mathbf{c}} = \rho_{\mathbf{s}, \mathbf{c}}(\mathbf{x}) / \rho_{\mathbf{s}, \mathbf{c}}(V)$.

2.2. Lattice

Let $B \in \mathbb{R}^{n \times n}$ where the column vectors $b_1, \dots, b_n$ are linearly independent. We refer to integer linear combinations of b_i as an n -dimensional full-rank lattice, denoted by $\Lambda(B) = \{\sum_{i=1}^n c_i b_i : c_i \in \mathbb{Z}\}$.

Definition 1 (NTRU Lattices). Let $d = 2^\kappa$ for some positive integer κ , $q \in \mathbb{Z}^+$ be a modulus. For $f, g \in \mathcal{R}$ with f invertible over $\mathcal{R}_q$, define $h = gf^{-1} \in \mathcal{R}_q$. The associated NTRU lattice is defined as

$$\Lambda_{h,q} = \{(u, v) \in \mathcal{R} \times \mathcal{R} : u + vh = 0 \bmod q\}.$$

Then, $\Lambda_{h,q}$ is a $2d$ -dimensional full-rank lattice generated by the rows of

$$B = \begin{bmatrix} -\text{rot}(h) & I_d \\ q \cdot I_d & 0 \end{bmatrix} \in \mathbb{Z}^{2d \times 2d}.$$

where I_d is the d -dimensional identity matrix.

Trapdoor Functions

Definition 2. [27,28] $\text{NTRU.TpGen}(q, d) \rightarrow (h, \mathbf{B})$: Given modulus q and ring dimension d , the algorithm outputs $h \in \mathcal{R}_q$ and a short basis $\mathbf{B}$ of the NTRU lattice $\Lambda_{h,q}$ satisfying the bound: $\|\tilde{\mathbf{B}}\| \leq 1.17\sqrt{q}$ where $\tilde{\mathbf{B}}$ denotes the Gram-Schmidt orthogonalized basis of B .

Definition 3. [27] $\text{GSamp}(h, \mathbf{a}, B, \mathbf{s}, c)$: Given $(h, B) \leftarrow \text{NTRU.TpGen}(q, d)$, $\mathbf{a} \in \mathcal{R}_q^m$, a standard deviation $\mathbf{s} > 0$ and a target vector $c \in \mathcal{R}_q$, the algorithm outputs a triple $(u, v, \mathbf{w})$ that satisfies $u + vh + \mathbf{a}^\top \mathbf{w} = c \bmod q$ and is distributed according to a discrete Gaussian distribution.

Difficult Problems

Definition 4 (The $\text{MSIS}_{n,m,B}$ Problem). Let an integer modulus $q > 0$ be fixed. Given $A \leftarrow \mathcal{R}_q^{n \times m}$, the Module-SIS problem with parameters $n, m > 0$ and $0 < B < q$ asks to find $\mathbf{s} \in \mathcal{R}_q^m$ such that $A\mathbf{s} = \mathbf{0} \bmod \mathcal{R}_q$ and $0 < \|\mathbf{s}\| \leq B$. An algorithm $\mathcal{A}$ is said to have an advantage ϵ in solving $\text{MSIS}_{n,m,B}$ if

$$\text{Adv}_{n,m,B}^{\text{MSIS}}(\mathcal{A}) = \Pr \left[\begin{array}{l} 0 < \|\mathbf{s}\| \leq B \wedge A\mathbf{s} = \mathbf{0} \\ A \leftarrow \mathcal{R}_q^{n \times m}, \mathbf{s} \leftarrow \mathcal{A}(A) \end{array} \right] \geq \epsilon.$$

Definition 5 (The $\text{MLWE}_{n,m,\chi,q}$ Problem). The Module-LWE problem with $m \geq n > 0$, a modulus q and a distribution χ over $\mathcal{R}$ asks the adversary $\mathcal{A}$ to distinguish between the uniform samples $(A, \mathbf{b}) \leftarrow \mathcal{R}_q^{n \times m} \times \mathcal{R}_q^n$ and $(A, \mathbf{b} = A\mathbf{s} + \mathbf{e})$ for $A \leftarrow \mathcal{R}_q^{n \times m}$, secret vector $\mathbf{s} \leftarrow \chi^m$ and error vector $\mathbf{e} \leftarrow \chi^n$. Then, $\mathcal{A}$ is said to have an advantage ϵ in solving $\text{MLWE}_{n,m,\chi,q}$ if

$$\text{Adv}_{n,m,\chi,q}^{\text{MLWE}}(\mathcal{A}) = \left| \Pr \left[\mathcal{A}(A, A\mathbf{s} + \mathbf{e}) = 1 \mid A \leftarrow \mathcal{R}_q^{n \times m}, \mathbf{s} \leftarrow \chi^m, \mathbf{e} \leftarrow \chi^n \right] - \Pr \left[\mathcal{A}(A, \mathbf{b}) = 1 \mid A \leftarrow \mathcal{R}_q^{n \times m}, \mathbf{b} \leftarrow \mathcal{R}_q^n \right] \right| \geq \epsilon.$$

Definition 6 (The Interactive NTRU-ISIS_f Problem [19]). Let $\text{pp} = (q, d, n, m, \ell_m, \ell_r, t, \mathbf{s}, B_s, B_m)$ be a parameter tuple. Let the preimage oracle $\mathcal{O}_{\text{pre}}(\mathbf{m}, \mathbf{r})$ be defined as follows: Given an input $(\mathbf{m}_i, \mathbf{r}_i)$ satisfying $\|(\mathbf{m}_i, \mathbf{r}_i)\| \leq B_m$, the oracle operates as follows. It first samples $x_i \leftarrow [2^t]$ and computes $s_i \leftarrow A_s^{-1}(f(x_i) + C \begin{bmatrix} \mathbf{m}_i \\ \mathbf{r}_i \end{bmatrix})$, and then returns (x_i, s_i) . Here, $A \in \mathcal{R}_q^{n \times (m+2)}$ and $C \in \mathcal{R}_q^{n \times (\ell_m + \ell_r)}$ are public random matrices, and $f : [2^t] \rightarrow \mathcal{R}_q$ is a fixed efficiently computable function.

In the interactive ISIS_f experiment $\text{Exp}_{\text{pp}}^{\text{Int-ISIS}_f}$, an adversary $\mathcal{A}$ queries $\mathcal{O}_{\text{pre}}$ up to k times. The adversary wins if it outputs a new tuple $(x^*, \mathbf{s}^*, \mathbf{m}^*, \mathbf{r}^*)$ satisfying $0 < \|\mathbf{s}^*\| \leq B_s$, $\|(\mathbf{m}^*, \mathbf{r}^*)\| \leq B_m$, $A\mathbf{s}^* = f(x^*) + C \begin{bmatrix} \mathbf{m}^* \\ \mathbf{r}^* \end{bmatrix}$ and $\mathbf{m}^* \notin \{\mathbf{m}_1, \dots, \mathbf{m}_k\}$. The Int-ISIS_f^{pp} assumption states that for any PPT adversary $\mathcal{A}$, the probability of winning this experiment is negligible. Formally,

$$\text{Adv}_{\text{pp}}^{\text{Int-ISIS}_f}(\mathcal{A}) = \Pr[\text{Exp}_{\text{pp}}^{\text{Int-ISIS}_f}(\mathcal{A}) \rightarrow 1]$$

is negligible.

The NTRU-ISIS_f problem is as hard as the standard MSIS_{1,m+2,B}, where

$$B = \frac{1}{2} \sqrt{(m+2)d} + B_m \psi d \sqrt{(\ell_m + \ell_r)(m+2)}.$$

2.3. Non-interactive zero-knowledge proofs

This section adopts the concept of non-interactive zero-knowledge proofs (NIZK) within the random-oracle model as introduced in [29]. Many efficient lattice-based NIZK constructions also adopt this paradigm.

Definition 7. Let R be a relation. A non-interactive zero-knowledge proof system for R is a tuple of PPT algorithms (Setup, Prove, Verify) defined as follows.

- **Setup**(1^λ): This algorithm outputs a common reference string crs .
- **Prove**(crs, x, w): This algorithm, given crs and a valid statement-witness pair $(x, w) \in R$, outputs a proof π or aborts with $\perp$.
- **Verify**(crs, x, π): This algorithm, given crs , x , and π , outputs a decision bit indicating acceptance or rejection.

A secure NIZK system is required to satisfy the following fundamental properties:

- **Completeness:** For any common reference string $\text{crs} \leftarrow \text{Setup}(1^\lambda)$ and any valid $(x, w) \in R$, $\text{Prove}(\text{crs}, x, w)$ outputs $\perp$ only with negligible probability, and the verifier accepts the resulting proof with probability $1 - \text{negl}(\lambda)$.
- **Soundness:** For any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ can produce a statement $x \notin L_R$ and a proof π^* such that the verifier accepts is negligible in λ .
- **Zero-Knowledge:** There exists a simulator $S = (S_0, S_1)$, consisting of two PPT algorithms with shared state, such that for any PPT adversary $\mathcal{A}$, the view of $\mathcal{A}$ interacting with the real prover is computationally indistinguishable from its interaction with the simulator. That is,

$$\text{Adv}_R^{\text{ZK}}(\mathcal{A}) = |\Pr[1 \leftarrow \mathcal{A}^{\text{H,Prove}}(\text{crs})] - \Pr[1 \leftarrow \mathcal{A}^{S_0, S_1}(\text{crs})]| \leq \text{negl}[\lambda].$$

- **Multi-Proof Extractability:** If an adversary is able to produce multiple valid proofs on adaptively chosen statements under a simulated crs , then there exists a straight-line extractor that can recover the corresponding witnesses. The property consists of:

- **CRS Simulatability:** The simulated crs is computationally indistinguishable from a real one, i.e., for any PPT adversary $\mathcal{A}$,

$$\text{Adv}_{\Pi_{\text{NIZK}}}^{\text{crs}}(\mathcal{A}) = |\Pr[\text{crs} \leftarrow \text{Setup}(1^\lambda) : 1 \leftarrow \mathcal{A}^{\text{H}}(\text{crs})] - \Pr[(\text{c}\tilde{\text{r}}\text{s}, \tau) \leftarrow S_{\text{crs}}(1^\lambda) : 1 \leftarrow \mathcal{A}^{\text{H}}(\text{c}\tilde{\text{r}}\text{s})]| \leq \text{negl}[\lambda].$$

- **Straight-Line Extractability:** There exist constants e_1, e_2, c such that for any $Q_H, Q_S \in \text{Poly}(\lambda)$ and any PPT adversary $\mathcal{A}$ that makes at most Q_H random-oracle queries with

$$\Pr[(\text{c}\tilde{\text{r}}\text{s}, \tau) \leftarrow S_{\text{crs}}(1^\lambda), \{(x_i, \pi_i)\}_{i \in [Q_S]} \leftarrow \mathcal{A}^{\text{H}}(\text{c}\tilde{\text{r}}\text{s}) : \forall i \in [Q_S], \text{Verify}^{\text{H}}(\text{c}\tilde{\text{r}}\text{s}, x_i, \pi_i) = 1] \geq \mu(\lambda)$$

where $\mu(\lambda)$ is non-negligible. If the adversary can generate valid proofs with non-negligible success probability, then there exists an efficient extractor that, given the trapdoor used in CRS simulation, can recover witnesses with probability at least $\frac{1}{2} \cdot \mu(\lambda) - \text{negl}[\lambda]$, where $\mu(\lambda)$ is a non-negligible function. We have

$$\begin{aligned} & \Pr[(\text{c}\tilde{\text{r}}\text{s}, \tau) \leftarrow S_{\text{crs}}(1^\lambda), \{(x_i, \pi_i)\}_{i \in [Q_S]} \leftarrow \mathcal{A}^{\text{H}}(\text{c}\tilde{\text{r}}\text{s}), \\ & \quad w_i \leftarrow \text{MultiExtract}(Q_H, Q_S, 1/\mu, \text{c}\tilde{\text{r}}\text{s}, \tau, x_i, \pi_i)_{i \in [Q_S]} : \\ & \quad \forall i \in [Q_S], (\text{c}\tilde{\text{r}}\text{s}, x_i, w_i) \in R \wedge \text{Verify}^{\text{H}}(\text{c}\tilde{\text{r}}\text{s}, x_i, \pi_i)] \\ & \geq \frac{1}{2} \mu(\lambda) - \text{negl}[\lambda]. \end{aligned}$$

The extractor runs in time bounded by $Q_H^{e_1} \cdot Q_S^{e_2} \cdot \frac{1}{\mu(\lambda)^c} \cdot \text{poly}(\lambda)$.

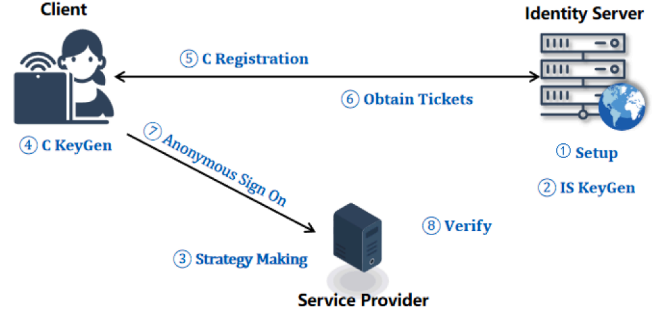


Fig. 1. ASSO architecture.

3. Definition of ASSO

In this section, we introduce the system model, formal definition, and security model for an anonymous single sign-on (ASSO) system.

3.1. System model

In this section, we introduce the ASSO system model, which supports selective attribute disclosure, designated verifiers, and token hiding. It consists of three entities: the client (C), the identity server (IS), and the service provider (SP), as shown in Fig. 1.

- **Identity server:** The IS sets up the system at the beginning (①) and generates its public and private keys (②). The IS is responsible for issuing certificates for the clients (⑤). If the credential is valid, the IS issues tickets to the client in accordance with the attribute disclosure policy (⑥).
- **Client:** C holds a device, such as a smartphone, and generates its unique attributes and private key (④). C registers with the IS to obtain an anonymous credential, which is issued based on C's attributes and private key (⑤). The credential is subsequently stored securely in the device. To obtain the tickets, C needs to show its credentials anonymously to the IS and reveal a particular subset of the attribute sets to prove that it satisfies the attribute disclosure policy (⑥). To access the specified service using the tickets, C calculates the access token by matching the attribute sets with the policy of the SP (⑦).
- **Service provider:** The SP is required to generate its public and private keys, formulate attribute access policies (③), verify whether C's submitted access token is valid, and then provide services to C (⑧).

Our ASSO scheme is described as follows. The IS sets up the system parameters (①), and generates its key pair (②). The SP formulates access policies for login and publishes them (③). When joining the SSO system, C generates its own private key, attribute set, and corresponding public key (④). C initiates a request to the IS to obtain a credential for C's public key, and the IS responds with a signature for C's public key (⑤). After successful registration, the IS stores the client's registration information, and C securely saves the credential on its device (⑥). To prevent replay attacks, when the client needs to request services from the SP, C obtains tickets that comply with the access policy from the IS (⑥). The client logs into the SP's service system anonymously by generating a token for each ticket (⑦). Finally, the SP verifies the token's validity and provides the corresponding services based on the tickets (⑧).

3.2. Formal definition

The formal definitions of the algorithms used in an SSO system with anonymous credentials are provided below.

- ① **Setup**($1^\lambda, 1^l$) $\rightarrow (pp, \mathcal{L})$: This algorithm is invoked by the IS. This algorithm takes a security parameter λ and a number of attributes l as input, and outputs the system tuple of parameters pp and the list of clients' registrations $\mathcal{L}$. For conciseness, pp will be omitted from the algorithms below and treated as an implicit parameter.

- ② **ISKeyGen**(1^λ) $\rightarrow (ipk, isk)$: This algorithm is invoked by the IS. This algorithm takes a security parameter λ as input and outputs the IS's public key ipk and private key isk .
- ③ **SPKeyGen**(1^λ) $\rightarrow (vpk, usk, idx)$: This algorithm generates the public and private keys $\{vpk, usk\}$ required for encrypting and decrypting the token, as well as the designated access policy idx , and is executed by the SP.
- ④ **CKeyGen**($1^\lambda, attrs$) $\rightarrow (upk, usk, attrs)$: This algorithm, executed by C, accepts a security parameter λ and the attribute set $attrs$ as input and outputs the private key usk and the corresponding public key upk for C.
- ⑤ **CReg**($C(upk, usk, attrs) \leftrightarrow IS(ipk, isk, L)$) $\rightarrow (ucred, L')/\perp$: This algorithm is jointly executed by C and IS using their respective inputs $(upk, usk, attrs)$ and (ipk, isk) . It returns C's credential $ucred$ and the updated client registration list L' on success or $\perp$ on failure.
- ⑥ **ObtTkt**($C(ucred, usk, attrs, idx) \leftrightarrow IS(ipk, isk)$) $\rightarrow (\pi_{cred}, tkt, dcid, v)/\perp$: This algorithm is jointly executed by the IS and C. C provides inputs $ucred, usk, attrs$, and the disclosed attribute index idx , while IS contributes ipk, isk . Upon successful execution, it returns a proof π_{cred} of $ucred$, a ticket tkt , its dual cost identifier $dcid$, and a validity period v ; failure results in $\perp$.
- ⑦ **SignOn**(tkt, v, ipk, vpk) $\rightarrow tok/\perp$: This algorithm, executed by C, generates an access token tok . This algorithm inputs tkt , the validity period v , and its identifiers $dcid$, as well as ipk and vpk , and outputs either a valid access token tok or the failure symbol $\perp$.
- ⑧ **Verify**(tok, ipk, vsk) $\rightarrow 1/0$: This algorithm is executed by the SP. This algorithm verifies the validity of tok . Upon successful verification, the algorithm returns 1 and SP grants service access to C; otherwise, it returns 0.

Remark 1. The identifier $dcid$ serves as a fresh unique identifier for the issued ticket and is used to prevent replay and double-spending attacks. The IS and SP maintain an authenticated append-only log of issued/spent pseudonymous tags. During ticket issuance, the IS assigns a unique $dcid$ to the ticket and records (t, v) in the issued log, where t denotes the encoded hash value of $dcid$. During token redemption, after verifying the validity of the token, the SP checks whether the corresponding $dcid$ -tag has already appeared in the spent log. If it has not appeared before, the SP accepts the token and atomically inserts the tag into the spent log; otherwise, the token is rejected as a replay or double-spending attempt.

Correctness. Correctness of the ASSO system is defined by whether the system properly validates all access tokens generated by legal clients. That is

$$\Pr \left\{ \begin{array}{l} \text{Setup}(1^\lambda, 1^l) \rightarrow (pp, L) \\ \text{ISKeyGen}(1^\lambda) \rightarrow (ipk, isk) \\ \text{SPKeyGen}(1^\lambda) \rightarrow (vpk, usk, idx) \\ \text{CKeyGen}(1^\lambda, attrs) \rightarrow (upk, usk, attrs) \\ \text{CReg}(C(upk, usk, attrs) \leftrightarrow IS(ipk, isk)) \rightarrow (ucred, L') \\ \text{ObtTkt}(C(ucred, usk, attrs, idx) \leftrightarrow IS(ipk, isk)) \rightarrow (\pi_{cred}, tkt, dcid, v) \\ \text{SignOn}(tkt, v, ipk, vpk) \rightarrow tok \end{array} \right\} = 1.$$

3.3. Security model

In this security paradigm, the IS operates with an honest-but-curious behavior. It honestly generates its key pair and issues registration certificates to clients that have committed their own attributes, while simultaneously attempting to infer undisclosed attributes and the client's private key. Considering that the clients are malicious, they may obtain the ticket illegally by forging credentials, or they may forge an access

token to access the designated service. Meanwhile, the SP is also honest but curious. That is, it verifies that the client's access token is valid but attempts to obtain private information, such as the client's private key.

An ASSO system must satisfy the unlinkability and unforgeability of credentials and access tokens. The system we propose also needs to satisfy the requirement of token hiding. We formalize a security game between a PPT adversary $\mathcal{A}$ and a challenger $\mathcal{C}$ to characterize the security model of our proposed system. $\mathcal{C}$ runs **Setup**, **ISKeyGen** and **SPKeyGen** to obtain $(pp, L, ipk, isk, vpk, usk, idx)$. $\mathcal{C}$ sends (pp, ipk, vpk) to the adversary $\mathcal{A}$.

Before $\mathcal{A}$ accesses the oracles, $\mathcal{C}$ initializes several lists $\mathcal{L}_{HC}, \mathcal{L}_{CC}, \mathcal{L}_{TKT}, \mathcal{L}_{TOK}$: $\mathcal{L}_{HC}$ stores $(upk, usk, attrs)$ of honest clients, $\mathcal{L}_{CC}$ stores upk of corrupted clients, $\mathcal{L}_{TKT}$ stores $(upk, attrs_{idx}, \pi_{cred}, tkt, dcid, v)$ when the IS issues tickets for the honest client, and $\mathcal{L}_{TOK}$ stores $\{(upk, attrs_{idx}, tok, vpk)\}$ when C sets up tokens for the SP.

- $\mathcal{O}_{RegC}(upk, attrs)$: This oracle simulates the issuance of registration credentials for an honest C with the attribute set $attrs$. It first executes $\{upk, usk, attrs\} \leftarrow \text{CKeyGen}(1^\lambda, attrs)$. If $(attrs, upk, *) \in \mathcal{L}_{HC} \cup \mathcal{L}_{CC}$, return $\perp$, otherwise, $(ucred, L') \leftarrow \text{CReg}(C(upk, usk, attrs) \leftrightarrow IS(ipk, isk, L))$. Then update the list $\mathcal{L}_{HC} := \mathcal{L}_{HC} \cup \{(upk, usk, attrs, ucred)\}$.
- $\mathcal{O}_{CorC}(upk)$: This oracle is used to allow an adversary $\mathcal{A}$ to control an honest client's upk . If $upk \in \mathcal{L}_{CC}$, it returns $\perp$. If $upk \in \mathcal{L}_{HC}$, it is removed from $\mathcal{L}_{HC}$ and added to $\mathcal{L}_{CC}$. Finally, it returns $(usk, attrs, ucred)$.
- $\mathcal{O}_{Obt}(upk, attrs_{idx})$: This oracle simulates the IS issuing tickets honestly for honest clients. If $upk \in \mathcal{L}_{CC}$, return $\perp$. Otherwise, it executes $(\pi_{cred}, tkt, dcid, v) \leftarrow \text{ObtTkt}(C(upk, usk, attrs, idx, ucred) \leftrightarrow IS(ipk, isk))$. Then, update the list $\mathcal{L}_{TKT} = \mathcal{L}_{TKT} \cup \{(upk, attrs_{idx}, \pi_{cred}, tkt, dcid, v)\}$.
- $\mathcal{O}_{ObtC}(upk, attrs_{idx})$: This oracle simulates a curious IS to issue tickets for honest clients. If $(upk, *) \in \mathcal{L}_{CC}$, return $\perp$, otherwise, $(\pi_{cred}, tkt, dcid, v) \leftarrow \text{ObtTkt}(C(upk, usk, attrs, idx, ucred) \leftrightarrow \mathcal{A}(*, ipk))$. Then, update the list $\mathcal{L}_{TKT} = \mathcal{L}_{TKT} \cup \{(upk, attrs_{idx}, \pi_{cred}, tkt, dcid, v)\}$. Here, $\mathcal{A}$ plays the role of the IS. Here, $*$ indicates an unknown key.
- $\mathcal{O}_{ObtIS}(upk, attrs_{idx})$: This oracle simulates an honest IS to issue tickets for malicious clients. If $(upk, *) \notin \mathcal{L}_{CC}$, return $\perp$, otherwise, $(\pi_{cred}, tkt, dcid, v) \leftarrow \text{ObtTkt}(\mathcal{A}(*, attrs_{idx}) \leftrightarrow IS(ipk, isk))$. Then, update the list $\mathcal{L}_{TKT} = \mathcal{L}_{TKT} \cup \{(upk, attrs_{idx}, \pi_{cred}, tkt, dcid, v)\}$. Here, $\mathcal{A}$ plays the role of the client.
- $\mathcal{O}_{SO}(upk, attrs_{idx}, tkt, vpk)$: This oracle simulates the honest C generating access tokens for the specified SP. If $(upk, *) \in \mathcal{L}_{CC}$, return $\perp$, otherwise, return $tok \leftarrow \text{SignOn}(tkt, v, ipk, vpk)$. Then, update the list $\mathcal{L}_{TOK} = \mathcal{L}_{TOK} \cup \{(upk, attrs_{idx}, tok, vpk)\}$.
- $\mathcal{O}_{UnICred_b}(upk_0, upk_1, idx)$: This oracle models a curious IS controlled by $\mathcal{A}$ attempting to distinguish credential proofs π_{ucred} for two distinct honest clients. The oracle takes as input $\{upk_0, upk_1\}$, which are two different honest clients, and the disclosed attribute-index set idx . Upon invocation, it executes the **ObtTkt** to generate (π_b, tkt_b) and returns π_b for $b \in \{0, 1\}$. For $\forall i \in idx$, the attribute values satisfy $|attrs_0| = |attrs_1|$ and $attrs_0[i] = attrs_1[i]$.
- $\mathcal{O}_{UnITok_b}(upk_0, upk_1, idx)$: This oracle models a curious SP controlled by $\mathcal{A}$ attempting to distinguish tokens for two distinct honest clients: it takes as input $\{upk_0, upk_1\}$ and disclosed attribute-index set idx , it first executes **ObtTkt** to generate tkt_b for $b \in \{0, 1\}$, then invokes **SignOn** to produce tok_b for tkt_b , and returns tok_b .
- $\mathcal{O}_{TokHid_b}(\{tkt_0, dcid_0, v_0\}, \{tkt_1, dcid_1, v_1\})$: This oracle models a non-designated SP controlled by $\mathcal{A}$ attempting to distinguish tokens derived from two different tickets. It takes $\{tkt_0, dcid_0, v_0\}, \{tkt_1, dcid_1, v_1\}$ as input, executes **SignOn** to generate tok_b for $b \in \{0, 1\}$, and returns tok_b .

One-more unforgeability of the client's credential: Mainly considering the malicious client, the one-more unforgeability of the credential means that for any PPT adversary $\mathcal{A}$, even if she has the disclosed attributes, she still cannot obtain a legitimate certificate $ucred$.

Definition 8. The ASSO system provides one-more unforgeability for client credentials if no PPT adversary $\mathcal{A}$ has a non-negligible advantage against $\mathcal{C}$ in Game_{ufc} defined below.

- **Setup:** $\mathcal{C}$ runs the **Setup**, **ISKeyGen** and **SPKeyGen** algorithms, then sends (pp, ipk) to $\mathcal{A}$, and initializes $\mathcal{L}_{HC}, \mathcal{L}_{CC}, \mathcal{L}_{TKT}, \mathcal{L}_{TOK} = \emptyset$.
- **Queries:** $\mathcal{C}$ plays the role of the IS and $\mathcal{A}$ plays the role of C. $\mathcal{A}$ queries oracles $\mathcal{O}_{RegC}, \mathcal{O}_{CorC}$ to obtain the client registration credential $ucred$ corresponding to the $(attr, idx, *)$ and the registration list $\mathcal{L}$. $\mathcal{A}$ may query oracles $\mathcal{O}_{Obt}, \mathcal{O}_{ObtS}$ to verify the credentials. If it succeeds, $\mathcal{A}$ increments Q_{Reg} and updates $\mathcal{L}_{CC}$ by adding $\{(attr, idx, *)\}$.
- **Output:** If $\mathcal{A}$ outputs $Q_{Reg} + 1$ valid $\{(attr_i, idx_i, ucred_i)\}_{i \in [Q_{Reg}+1]}$ such that any two pairs $\{(attr_i, idx_i)\}$ differ in at least one common attribute, and verification is successful, $\mathcal{A}$ wins the game.

Unforgeability of tokens: Mainly considering the malicious client, the unforgeability of the token means that for any PPT adversary $\mathcal{A}$, even if she has the disclosed attributes, she cannot forge the access token tok for a ticket that is not issued by the IS.

Definition 9.

The ASSO system provides token unforgeability if no PPT adversary $\mathcal{A}$ has a non-negligible advantage against $\mathcal{C}$ in Game_{uft} defined below.

- **Setup:** $\mathcal{C}$ runs the **Setup**, **ISKeyGen** and **SPKeyGen** algorithms, then sends (pp, ipk) to $\mathcal{A}$, and initializes $\mathcal{L}_{HC}, \mathcal{L}_{CC}, \mathcal{L}_{TKT}, \mathcal{L}_{TOK} = \emptyset$.
- **Queries:** $\mathcal{C}$ plays the role of IS and $\mathcal{A}$ plays the role of C. $\mathcal{A}$ may query oracles $\mathcal{O}_{RegC}, \mathcal{O}_{CorC}$ to obtain the client registration credential $ucred$ corresponding to the $(attr, idx, *)$ and the registration list $\mathcal{L}$. $\mathcal{A}$ may query the oracle $\mathcal{O}_{Obt}$ to obtain the ticket. Then, $\mathcal{A}$ may query the oracle $\mathcal{O}_{SO}$ to obtain a token.
- **Output:** If $\mathcal{A}$ outputs $(upk^*, attr_{idx}^*, tok^*)$ such that verification accepts, the ticket identifier/signature embedded in tok^* was not generated by $\mathcal{O}_{Obt}$, and $tok^* \notin \mathcal{L}_{TOK}$, then $\mathcal{A}$ wins.

Unlinkability of the client's credential: Mainly considering the honest-but-curious IS, the unlinkability of the client's credential means that for any PPT adversary $\mathcal{A}$, she cannot distinguish the credentials showing proof of the two honest clients at **ObtTkt** phase.

Definition 10. The ASSO system achieves unlinkability of client credentials if no PPT adversary $\mathcal{A}$ has a non-negligible advantage against $\mathcal{C}$ in the Game_{ulc} defined below.

- **Setup:** $\mathcal{C}$ runs the **Setup**, **ISKeyGen** and **SPKeyGen** algorithms, then sends (pp, ipk, idx) to $\mathcal{A}$, and initializes $\mathcal{L}_{HC}, \mathcal{L}_{CC}, \mathcal{L}_{TKT}, \mathcal{L}_{TOK} = \emptyset$.
- **Queries:** $\mathcal{C}$ plays the role of C and $\mathcal{A}$ plays the role of IS. $\mathcal{A}$ may query oracles $\mathcal{O}_{RegC}, \mathcal{O}_{CorC}, \mathcal{O}_{Obt}, \mathcal{O}_{ObtC}$ and $\mathcal{O}_{UnICred_b}$.
- **Output:** If $\mathcal{A}$ outputs $b^* = b$, $\mathcal{A}$ wins the game.

Unlinkability of the tokens: Mainly considering the honest-but-curious SP, the unlinkability of the tokens means that for any PPT adversary $\mathcal{A}$, she cannot distinguish the tokens of the two honest clients at the **Verify** phase.

Definition 11. The ASSO system achieves unlinkability of client tokens if no PPT adversary $\mathcal{A}$ has a non-negligible advantage against $\mathcal{C}$ in Game_{ult} defined below.

- **Setup:** $\mathcal{C}$ runs the **Setup**, **ISKeyGen** and **SPKeyGen** algorithms, then sends (pp, ipk, idx) to $\mathcal{A}$, and initializes $\mathcal{L}_{HC}, \mathcal{L}_{CC}, \mathcal{L}_{TKT}, \mathcal{L}_{TOK} = \emptyset$.
- **Queries:** $\mathcal{C}$ plays the role of C and $\mathcal{A}$ plays the role of SP. $\mathcal{A}$ may query oracles $\mathcal{O}_{RegC}, \mathcal{O}_{CorC}, \mathcal{O}_{Obt}, \mathcal{O}_{SO}$ and $\mathcal{O}_{UnITok_b}$.
- **Output:** If $\mathcal{A}$ outputs $b^* = b$, $\mathcal{A}$ wins the game.

Token-hiding: Mainly considering the non-designated SP, token-hiding means that any non-designated SP cannot distinguish the tokens of two different tickets at the **SignOn** phase.

Definition 12. The ASSO system achieves token hiding if no PPT adversary $\mathcal{A}$ has a non-negligible advantage against $\mathcal{C}$ in Game_{th} defined below.

- **Setup:** $\mathcal{C}$ runs the **Setup**, **ISKeyGen** and **SPKeyGen** algorithms, then sends (pp, ipk) to $\mathcal{A}$, and initializes $\mathcal{L}_{HC}, \mathcal{L}_{CC}, \mathcal{L}_{TKT}, \mathcal{L}_{TOK} = \emptyset$.
- **Queries:** $\mathcal{C}$ plays the role of C and $\mathcal{A}$ plays the role of SP. $\mathcal{A}$ may query oracles $\mathcal{O}_{RegC}, \mathcal{O}_{CorC}, \mathcal{O}_{Obt}, \mathcal{O}_{SO}$ and $\mathcal{O}_{TokHid_b}$.
- **Output:** If $\mathcal{A}$ outputs $b^* = b$, $\mathcal{A}$ wins the game.

4. Proposed lattice-based single sign-on system

4.1. Technical overview

The proposed system utilizes the following building blocks: a blind signature scheme based on NTRU-ISIS_f [19], a public-key encryption scheme Kyber [30], a symmetric encryption algorithm AES [31], and two NIZK systems [24] for the NP-relations R^{ISIS} and R^{Sign} defined below.

The construction works as follows. The IS is associated with a signing-verification key pair $\{ipk, isk\}$ generated according to NTRU-ISIS_f. The public-private key pairs $((a_1, a_2), B)$ and (h, B_{kt}) are both generated via the NTRU trapdoor generation algorithm [28], where the former is used for issuing and verifying registration certificates, and the latter is used for issuing and verifying tickets. The public keys (c_0, c_1, c_2) are generated to create the client's public key and hide the ticket identifier. The SP is associated with an encryption-decryption key pair (vpk, usk) for Kyber. The client's private key usk is randomly sampled, and the public key upk is computed from usk and attributes $\mathbf{m}$ via $upk = \mathbf{c}_0^T \begin{bmatrix} usk \\ \mathbf{m} \end{bmatrix} + \mathbf{c}_1^T \mathbf{r}$.

In our system, the client needs to register by constructing NIZK proofs for registration. The registration proof uses R^{ISIS} defined in Eq. (1).

$$R^{\text{ISIS}} = \left\{ \begin{array}{l} \mathbf{x} = (A, \bar{\mathbf{u}}, B_s) \\ \mathbf{w} = \bar{\mathbf{s}} \end{array} \middle| \begin{array}{l} A \in \mathbb{Z}_q^{n \times m}, \bar{\mathbf{u}} \in \mathbb{Z}_q^n, \bar{\mathbf{s}} \in \mathbb{Z}_q^m : \\ A\bar{\mathbf{s}} = \bar{\mathbf{u}} \pmod{q} \wedge \|\bar{\mathbf{s}}\| \leq B_s \end{array} \right\} \quad (1)$$

$$R^{\text{Sign}} = \left\{ \begin{array}{l} \mathbf{x} = (A, B, C, \bar{\mathbf{m}}, B_s, B_r) \\ \mathbf{w} = (\bar{\mathbf{s}}, \bar{\mathbf{r}}, \bar{\mathbf{u}}) \end{array} \middle| \begin{array}{l} (A, B, C) \in \mathbb{Z}_q^{n \times (m + t + (\ell_m + \ell_r))}, \bar{\mathbf{m}} \in \mathbb{Z}_q^{\ell_m}, \bar{\mathbf{u}} \in \{0, 1\}^t : \\ A\bar{\mathbf{s}} = B\bar{\mathbf{u}} + C \begin{bmatrix} \bar{\mathbf{m}} \\ \bar{\mathbf{r}} \end{bmatrix} \wedge \|\bar{\mathbf{s}}\| \leq B_s \wedge \|\bar{\mathbf{r}}\| \leq B_r \end{array} \right\} \quad (2)$$

When registering, the client generates a NIZK argument π_C for the relation R^{ISIS} where $A = \text{rot}(\begin{bmatrix} \mathbf{c}_0^T & \mathbf{c}_1^T \end{bmatrix}), \bar{\mathbf{u}} = \text{Coeffs}(upk), B_s = \psi \sqrt{(\ell_m + \ell_r)d}, \bar{\mathbf{s}} = \text{Coeffs}(usk, \mathbf{m}, \mathbf{r})$. The client sends the public-key/proof pair (upk, π_C) to the IS. If the verification is successful, then the IS generates a signature (σ_C, x_C) for upk using the key pair $((a_1, a_2), B)$ such that $[a_1 | a_2 | 1] \sigma_C = f(x_C) + upk$. The client combines the signature with the client's randomness $\mathbf{r}$ to form the registration credential $cred$.

When requesting a ticket, the client needs to construct an NIZK proof for the relation R^{Sign} defined in Eq. (2). The client sends the credential proof π_{cred} to the IS, where $A = \text{rot}([a_1 | a_2 | 1]), C = \text{rot}(\begin{bmatrix} (\mathbf{c}_0^T)_{\text{idx}} & (\mathbf{c}_0^T)_{\{0, \dots, \text{idx}\}} & \mathbf{c}_1^T \end{bmatrix}), \bar{\mathbf{m}} = \text{Coeffs}(\mathbf{m})_{\text{idx}}, \bar{\mathbf{s}} = \text{Coeffs}(\sigma_C), \bar{\mathbf{u}} = \text{enc}(x_C), \bar{\mathbf{r}} = \text{Coeffs}(s, \mathbf{m}_{\text{idx}}, \mathbf{r}), B_s = \delta \sqrt{(m+2)d}, B_r = \psi \cdot \sqrt{h \cdot (|\text{idx}| + 1) + \ell_r d}$. If the verification is successful, then IS signs the encoded ticket identifier $\mathbf{t}$ together with the validity period v using (h, B_{kt}) such that $[h | 1] \sigma_{\text{kt}} = f(v) + \mathbf{c}_2^T \mathbf{t}$ and forms the ticket tk .

When signing on, the client generates a NIZK argument tok' for the relation R^{ISIS} defined in Eq. (1) over $\mathbb{Z}_p$, where $A = \text{rot}([h | 1]), \bar{\mathbf{s}} = \text{Coeffs}(\sigma_{\text{kt}}), B_s = \delta' \sqrt{2d'}, \bar{\mathbf{u}} = \text{Coeffs}(f(v) + \mathbf{c}_2^T \mathbf{t})$. Then a random AES key K is generated. SP's public key vpk is used to encrypt K , and K is used to encrypt $\{\text{tok}', \mathbf{t}, v\}$. Verification is performed after decrypting tok with usk .

The correctness of our LAASSO is based on the correctness and security of the underlying building blocks. The signature algorithm ensures

Table 1
Explanation of notations and parameters.

Notation	Explanation
q, p	Modulus
d	Degree of the cyclotomic ring $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^d + 1)$
d'	Degree of the cyclotomic ring $\mathcal{R}_p = \mathbb{Z}_p[x]/(x^{d'} + 1)$
$m + \ell_m + \ell_r + \ell_t$	Length of (a_2, c_0, c_1, c_2)
t, t'	Length of $\text{enc}(x)$
N	Domain size of the function $f, N = 2^t$
h	Length of the string output by H_M
k	Length of the string output by H_L , satisfies $k = d' \cdot \ell_t$
l	The number of attributes, satisfies $d \mid (l + 1)h$
$\text{idx}/\overline{\text{idx}}$	The index set of disclosed/undisclosed attributes
$ \text{idx} = l'$	Length of the index set of disclosed attributes
$\mathfrak{s}$	Preimages are sampled from $D_{\mathfrak{s}}$
ψ	Infinity norm of $\mathbf{r}$ and the output of H_M
B	Random matrix corresponding to $f(x)$
param	The parameter required by the NIZK proofs
crs	The common random string required by the NIZK proofs

the unforgeability of our system. Zero-knowledge proof systems for attribute policy verification provide privacy protection for our system. The public-key and symmetric encryption schemes are used to achieve designated-verifier and token-hiding. Additionally, adversaries may intercept legitimate credentials and then resend them to gain unauthorized access, a technique known as a replay attack. Therefore, our system requires the client to request a ticket from the IS before obtaining the service. To prevent double-spending attacks, we introduce a unique ticket identifier and a validity period.

4.2. Description of LAASSO

LAASSO can be decomposed into five distinct phases: global setup, key generation, registration, sign-on, and verification. For simplicity, we represent the zero-knowledge proof and verification algorithms in Definition 7 as $\text{Prove}(w)$ and $\text{Verify}(\pi)$. We will present a more detailed illustration in Section 4.3.

Phase 1 Global Setup: The system initializes public parameters and secret keys.

Setup $(1^\lambda, 1^l) \rightarrow (pp, \mathcal{L})$: Based on the security parameter λ and the number of attributes l , the parameters are formally defined and presented in Table 1.

Let $f(x) := \text{Coeffs}^{-1}(B \cdot \text{enc}(x)) \in \mathcal{R}_q$ be a fixed function, where $B \in \mathbb{Z}_q^{d \times t}$, $\text{enc}(x)$ is the binary decomposition of $x - 1$.

Select hash functions: $H_M : \{0, 1\}^* \rightarrow [-\psi, \psi]^h$, $H_L : \{0, 1\}^* \rightarrow \{0, 1\}^k$.

The IS also instantiates a client registration list $\mathcal{L} = \emptyset$.

Phase 2 Key Generation: This phase involves the following algorithms.

ISKeyGen $(1^\lambda) \rightarrow (ipk, isk)$: Select $a_1, \mathbf{B} \leftarrow \text{NTRU.TpGen}(q, d)$ and $h, \mathbf{B}_{ikt} \leftarrow \text{NTRU.TpGen}(p, d')$. Sample $\mathbf{a}_2 \xleftarrow{\$} \mathcal{R}_q^m$ and $(c_0, c_1) \xleftarrow{\$} \mathcal{R}_q^{\ell_m} \times \mathcal{R}_q^{\ell_r}$. Sample $c_2 \xleftarrow{\$} \mathcal{R}_q^{\ell_t}$. Let $ipk = (a_1, \mathbf{a}_2, c_0, c_1, h, c_2)$ and $isk = \{\mathbf{B}, \mathbf{B}_{ikt}\}$.

SPKeyGen $(1^\lambda) \rightarrow (vpk, vsk, \text{idx})$: Select $(vpk, vsk) \leftarrow \text{Kyber.KeyGen}(\lambda)$. Select an array $\text{idx} \in [l]^{l'}$, with $1 \leq l' \leq l$, consisting of l' pairwise distinct indices that correspond to the attributes in the array attrs which need to be disclosed.

CKeyGen $(1^\lambda, \text{attrs}) \rightarrow (upk, usk, \mathbf{m})$: Sample $s \xleftarrow{\$} S_\psi$ as the private key of C. Parse $\text{attrs} = \{a_i\}_{i \in [l]}$ as the client's attribute set, where $a_i \in \{0, 1\}^*$. Compute $\mathbf{m} = \text{Coeffs}^{-1}(H_M(a_1), \dots, H_M(a_l)) \in \mathcal{R}_q^{\ell_m-1}$, which represents the client's attribute vector. Sample $\mathbf{r} \xleftarrow{\$} S_\psi^{\ell_r}$, and compute $c = c_0^\top [s] + c_1^\top \mathbf{r} \in \mathcal{R}_q$. Let $upk = c$ and $usk = s$.

Phase 3 Registration: This phase includes the following algorithm.

CReg $(C(upk, usk, \text{attrs}) \leftrightarrow \text{IS}(ipk, isk, \mathcal{L})) \rightarrow (\text{ucred}, \mathcal{L}')/\perp$: As shown in Fig. 2, C first proves that upk is well formed: π_C . Upon verifying the validity of π_C , the IS generates a signature for upk and updates the list $\mathcal{L}'$ to reflect the completed enrollment. C securely stores the credential ucred .

Phase 4 Sign-on: This phase consists of the interaction between IS and C through the following algorithm.

ObtTkt $(C(\text{ucred}, usk, \text{attrs}, \text{idx}) \leftrightarrow \text{IS}(ipk, isk)) \rightarrow (\pi_{\text{cred}}, \text{tkt}, dcid, v)/\perp$: As shown in Fig. 2, C first proves π_{cred} regarding the undisclosed attributes and credential ucred , and then selects the validity period of the ticket v . After verifying the validity of π_{cred} , the IS selects a unique ticket identifier $dcid$ at random and encodes it as a polynomial vector $\mathbf{t}$.

SignOn $(\text{tkt}, v, ipk, vpk) \rightarrow \text{tok}/\perp$: As shown in Fig. 2, C calculates a proof tok' regarding the ticket. C hides the token using AES and Kyber, and designates the verifier. It sends $\text{tok} = (\text{tok}_1, \text{tok}_2)$ to the SP.

Phase 5 Verify: This phase includes the following algorithm.

Verify $(\text{tok}, ipk, vsk) \rightarrow 1/0$: The SP computes $K = \text{Kyber.Dec}(vsk, \text{tok}_1)$ and $(\text{tok}', \mathbf{t}, v) = \text{AES.Dec}(K, \text{tok}_2)$. If $\mathbf{t}$ indicates that it has been consumed or v has exceeded its validity period, then output 0. Then it verifies tok' by executing $\text{Verify}_{\text{ISIS}}(\text{tok}')$. If the verification is successful, output 1 and grant access to the requested services; otherwise, output 0.

4.3. NIZKPoK

LAASSO uses the following building blocks: NIZK system $\Pi_{\text{ISIS}} = (\text{Prove}_{\text{ISIS}}^H, \text{Verify}_{\text{ISIS}}^H)$, $\Pi_{\text{Sign}} = (\text{Prove}_{\text{Sign}}^H, \text{Verify}_{\text{Sign}}^H)$. We mainly use the LNP framework proposed by Lyubashevsky et al. [24], and follow the improved NIZK by Bootle et al. [19] to prove various lattice-related statements and provide multi-theorem straight-line extractability. More details can be obtained by referring to [19].

The NIZK proof system $\Pi_{\text{ISIS}} = (\text{Prove}_{\text{ISIS}}^H, \text{Verify}_{\text{ISIS}}^H)$ is constructed to prove the relation in Eq. (1) with a common random string crs over $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^d + 1)$.

In the registration phase, the statement $c = c_0^\top [s] + c_1^\top \mathbf{r}$ over $\mathcal{R}_q$ is equivalent to proving

$$\text{Coeffs}(c) = \text{rot}(c_0^\top) \text{Coeffs}([s]) + \text{rot}(c_1^\top) \text{Coeffs}(\mathbf{r})$$

over $\mathbb{Z}_q$. Thus, the client sets

$$\begin{aligned} A &= [\text{rot}(c_0^\top) | \text{rot}(c_1^\top)] \in \mathbb{Z}_q^{d \times (\ell_m + \ell_r)d}, \\ \vec{s} &= (\text{Coeffs}(s), \text{Coeffs}(\mathbf{m}), \text{Coeffs}(\mathbf{r})) \in \mathbb{Z}_q^{(\ell_m + \ell_r)d}, \\ \vec{u} &= \text{Coeffs}(c) \in \mathbb{Z}_q^d, \\ B_s &= \psi \sqrt{(\ell_m + \ell_r)d}. \end{aligned}$$

It is equivalent to proving $A\vec{s} = \vec{u}$. Since the NIZK algorithm works modulo $\hat{q}$, we instead prove that

$$(\frac{\hat{q}}{q} \cdot A) \vec{s} = (\frac{\hat{q}}{q} \cdot \vec{u}) \pmod{\hat{q}}.$$

We have

$$\pi_C \leftarrow \text{Prove}_{\text{ISIS}}^H(\text{crs}_{\text{ISIS}}, (\hat{q}/q \cdot A, \hat{q}/q \cdot \vec{u}, \psi \sqrt{(\ell_m + \ell_r)d}, \vec{s})).$$

The IS computes

$$\text{Verify}_{\text{ISIS}}^H(\text{crs}_{\text{ISIS}}, (\hat{q}/q \cdot A, \hat{q}/q \cdot \vec{u}, \psi \sqrt{(\ell_m + \ell_r)d}, \pi_C)).$$

CReg($C(upk, usk, attrs) \leftrightarrow IS(ipk, isk, \mathcal{L}) \rightarrow (ucred, \mathcal{L}')/\perp$)

1. **Client:** Prove $\pi_C \leftarrow \text{Prove}_{\text{ISIS}}(s, \mathbf{m}, \mathbf{r})$.
Send (c, π_C) to the IS.
2. **IS:** If $\text{Verify}_{\text{ISIS}}(\pi_C) = 0$, then return $\perp$. Otherwise, sample $x_C \xleftarrow{\$} [2^t]$ and compute $\sigma_C \leftarrow \text{GSamp}(a_1, a_2, \mathbf{B}, \mathbf{s}, f(x_C) + c)$.
Return (σ_C, x_C) and update $\mathcal{L}' = \mathcal{L} \cup \{c, \sigma_C, x_C\}$.
Send (σ_C, x_C) to C.
3. **Client:** If $(\|\sigma_C\| > \mathbf{s} \sqrt{(m+2)d}) \vee (\|\mathbf{r}\| > \psi \sqrt{\ell_r d}) \vee ([a_1 | a_2 | 1] \sigma_C \neq f(x_C) + c)$ then return $\perp$. Store $ucred = (\mathbf{r}, \sigma_C, x_C)$.

ObtTkt($C(ucred, usk, attrs, idx) \leftrightarrow IS(ipk, isk) \rightarrow (\pi_{\text{cred}}, \text{tkt}, dcid, v)/\perp$)

1. **Client:** Prove $\pi_{\text{cred}} \leftarrow \text{Prove}_{\text{Sign}}(ucred, \text{Coeffs}(\mathbf{m})_{\text{idx}})$.
Set the validity period $v \xleftarrow{\$} [2^{t'}]$. Send $(\text{Coeffs}(\mathbf{m})_{\text{idx}}, \pi_{\text{cred}}, v)$ to the IS.
2. **IS:** If $\text{Verify}_{\text{Sign}}(\pi_{\text{cred}}, \text{Coeffs}(\mathbf{m})_{\text{idx}}) = 0$, then return $\perp$.
Otherwise, select $dcid \xleftarrow{\$} \{0, 1\}^*$ and encode it as $\mathbf{t} = \text{Coeffs}^{-1}(H_L(dcid)) \in (\mathbb{Z}_2[x]/(x^{d'} + 1))^{\ell_t}$. Compute $\sigma_t \leftarrow \text{GSamp}(h, \mathbf{B}_{\text{tkt}}, \mathbf{s}', f(v) + \mathbf{c}_2^T \mathbf{t})$.
Send $(\sigma_t, \mathbf{t})$ to C.
3. **Client:** If $(\|\sigma_t\| > \mathbf{s}' \sqrt{2d'}) \vee ([h | 1] \sigma_t \neq f(v) + \mathbf{c}_2^T \mathbf{t})$ then return $\perp$. Store $\text{tkt} = (\mathbf{t}, \sigma_t, v)$.

SignOn($\text{tkt}, v, ipk, vpk) \rightarrow \text{tok}/\perp$)

Client: Prove $\text{tok}' \leftarrow \text{Prove}_{\text{ISIS}}(\sigma_t)$. Select $K \xleftarrow{\$} \{0, 1\}^\lambda$. Compute $\text{tok}_1 \leftarrow \text{Kyber.Enc}(vpk, K)$, $\text{tok}_2 \leftarrow \text{AES.Enc}(K, \{\text{tok}', \mathbf{t}, v\})$.
Send $\text{tok} = (\text{tok}_1, \text{tok}_2)$ to the SP.

Fig. 2. The registration and sign-on phases.

In the sign-on phase, $[h | 1] \sigma_{\text{tkt}} = f(v) + \mathbf{c}_2^T \mathbf{t}$ over $\mathcal{R}_p$ is equivalent to proving

$$\text{rot}([h | 1]) \cdot \text{Coeffs}(\sigma_{\text{tkt}}) = \text{Coeffs}(f(v)) + \text{rot}(\mathbf{c}_2^T) \cdot \text{Coeffs}(\mathbf{t})$$

over $\mathbb{Z}_p$. Since the verifier already knows $\mathbf{t}$ and v , it is equivalent to proving the linear relation

$$\left(\frac{\hat{q}}{p} \cdot A\right) \cdot \vec{s} = \frac{\hat{q}}{p} \cdot \vec{u} \pmod{\hat{q}},$$

where

$$\begin{aligned} A &= \text{rot}([h | 1]) \in \mathbb{Z}_p^{d' \times 2d'}, \vec{s} = \text{Coeffs}(\sigma_{\text{tkt}}) \in \mathbb{Z}_p^{2d'}, \\ \vec{u} &= \text{Coeffs}(f(v)) + \text{rot}(\mathbf{c}_2^T) \cdot \text{Coeffs}(\mathbf{t}) \in \mathbb{Z}_p^{d'}, \\ B_s &= \mathbf{s}' \sqrt{2d'}. \end{aligned} \quad (3)$$

The client proves $\text{tok}' \leftarrow \text{Prove}_{\text{ISIS}}^{\text{H}_{\text{ISIS}}}(\text{crs}_{\text{ISIS}}, (\hat{q}/p \cdot A, \vec{u}, B_s, \vec{s}))$. The SP computes $\text{Verify}_{\text{ISIS}}^{\text{H}_{\text{ISIS}}}(\text{crs}_{\text{ISIS}}, (\hat{q}/p \cdot A, \vec{u}, B_s, \text{tok}'))$.

The proof system $\Pi_{\text{Sign}} = (\text{Prove}_{\text{Sign}}^{\text{H}}, \text{Verify}_{\text{Sign}}^{\text{H}})$ is constructed to prove knowledge of signatures and undisclosed attributes. We define the relation in Eq. (2). Specifically, we need to prove that $[a_1 | a_2 | 1] \sigma = f(x) + c$ over $\mathcal{R}_q$ is equivalent to proving

$$\begin{aligned} &\text{rot}([a_1 | a_2 | 1]) \cdot \text{Coeffs}(\sigma) \\ &= \text{Coeffs}(f(x)) + \text{rot}(\mathbf{c}_0^T) \cdot \text{Coeffs}\left(\begin{bmatrix} s \\ \mathbf{m} \end{bmatrix}\right) + \text{rot}(\mathbf{c}_1^T) \cdot \text{Coeffs}(\mathbf{r}) \end{aligned}$$

over $\mathbb{Z}_q$. Furthermore, since

$$\begin{aligned} &\text{rot}(\mathbf{c}_0^T) \begin{bmatrix} \text{Coeffs}(s) \\ \text{Coeffs}(\mathbf{m}) \end{bmatrix} + \text{rot}(\mathbf{c}_1^T) \cdot \text{Coeffs}(\mathbf{r}) \\ &= \text{rot}(\mathbf{c}_0^T)_{\text{idx}} \text{Coeffs}(\mathbf{m})_{\text{idx}} + \left[\text{rot}(\mathbf{c}_0^T)_{\{0, \text{idx}\}} | \text{rot}(\mathbf{c}_1^T) \right] \begin{bmatrix} \text{Coeffs}(s) \\ \text{Coeffs}(\mathbf{m})_{\text{idx}} \\ \text{Coeffs}(\mathbf{r}) \end{bmatrix}, \end{aligned}$$

it is equivalent to proving the linear relation

$$\left(\frac{\hat{q}}{q} \cdot A\right) \cdot \vec{s} = \left(\frac{\hat{q}}{q} \cdot B\right) \cdot \vec{u} + \left(\frac{\hat{q}}{q} \cdot C\right) \begin{bmatrix} \vec{m} \\ \vec{r} \end{bmatrix} \pmod{\hat{q}},$$

where

$$\begin{aligned} A &= \text{rot}([a_1 | a_2 | 1]) \in \mathbb{Z}_q^{d \times (m+2)d}, \\ C &= \left[\text{rot}(\mathbf{c}_0^T)_{\text{idx}} | \text{rot}(\mathbf{c}_0^T)_{\{0, \text{idx}\}} | \text{rot}(\mathbf{c}_1^T) \right] \in \mathbb{Z}_q^{d \times (\ell_m + \ell_r)d}, \\ \vec{m} &= \text{Coeffs}(\mathbf{m})_{\text{idx}} \in \mathbb{Z}_q^{|\text{idx}|h}, \\ \vec{s} &= \text{Coeffs}(\sigma) \in \mathbb{Z}_q^{(m+2)d}, \vec{u} = \text{enc}(x) \in \{0, 1\}^t, \\ \vec{r} &= \text{Coeffs}(s, \mathbf{m}_{\text{idx}}, \mathbf{r}) \in \mathbb{Z}_q^{h + |\text{idx}|h + \ell_r d}, \\ B_s &= \mathbf{s} \sqrt{(m+2)d}, \quad B_r = \psi \sqrt{h \cdot (|\text{idx}| + 1) + \ell_r d}. \end{aligned} \quad (4)$$

In the ticket-obtaining phase, the client proves $\pi_{\text{cred}} \leftarrow \text{Prove}_{\text{Sign}}^{\text{H}_{\text{Sign}}}(\text{crs}_{\text{sign}}, \left(\frac{\hat{q}}{q} \cdot A, \frac{\hat{q}}{q} \cdot B, \frac{\hat{q}}{q} \cdot C, \vec{m}, B_s, B_r, \text{idx}\right), (\vec{s}, \vec{r}, \vec{u}))$.

The IS computes

$$\text{Verify}_{\text{Sign}}^{\text{H}_{\text{Sign}}}(\text{crs}_{\text{sign}}, \left(\frac{\hat{q}}{q} \cdot A, \frac{\hat{q}}{q} \cdot B, \frac{\hat{q}}{q} \cdot C, \vec{m}, B_s, B_r, \text{idx}\right), \pi_{\text{cred}}).$$

4.4. Correctness and security

Theorem 4.1. *If the NIZK protocols and the PKE scheme are correct, then our system is correct.*

Proof. We assume the global setup and the key generation phases are executed honestly and correctly. There exists an efficient $\text{NTRU.TpGen}(q, d)$ algorithm that generates the public key h and the basis $\mathbf{B}$, such that $\|\mathbf{B}\| \leq 1.17 \sqrt{q}$ [28]. During registration, the client generates zero-knowledge proofs for the attributes honestly. Given the correctness of the NIZK protocols, the IS will successfully verify the proof of knowledge, except with a negligible probability. The IS issues a signature σ that satisfies $[a_1 | a_2 | 1] \sigma = f(x) + c$ to the client. Because the distribution of σ follows a discrete Gaussian distribution [32], the signature satisfies

$\|\sigma\| \leq \tilde{s}\sqrt{(m+2)d}$ with an overwhelming probability. Furthermore, we can obtain $\|\mathbf{r}\| \leq \psi\sqrt{\ell_r d}$ by $\mathbf{r} \leftarrow S_{\psi}$. In **ObtTkt**, the client's credential-knowledge proof passes verification except with negligible probability due to the correctness of the NIZK protocol. We also have $\|\sigma_i\| \leq \tilde{s}'\sqrt{2d'}$ with overwhelming probability. Similarly, the correctness of the **SignOn** and **Verify** algorithms is guaranteed by the correctness of the NIZK protocols and the PKE scheme. $\square$

Theorem 4.2. *Our system satisfies the unlinkability of credentials if the NIZK protocols are zero-knowledge.*

Proof. Let $\mathcal{A}$ be any PPT adversary against our LAASSO scheme with advantage ϵ . We will prove $\epsilon = \text{negl}[\lambda]$ using the ZK property of the underlying NIZK systems. Specifically, we construct indistinguishable games G_b, G_2 such that G_b is the experiment $\text{Expt}_A^{\text{UnlCred}}(1^\lambda)$. Denote E_i as the event that G_i outputs 1 for $i \in \{0, 1, 2\}$.

Game G_b : This game is the original unlinkability of credentials experiment $\text{Expt}_A^{\text{UnlCred}-b}(1^\lambda)$ for $b \in \{0, 1\}$. $\mathcal{C}$ runs **Setup**, **ISKeyGen** and **SPKeyGen** algorithms, then sends (pp, ipk, idx) to $\mathcal{A}$. $\mathcal{A}$ has full control of the registration list $\mathcal{L}$ and system information during the lifetime of the scheme. $\mathcal{A}$ can introduce honest attributes and a private/public key pair to the system by interacting with the oracle $\mathcal{O}_{\text{RegC}}$, but it is required to guarantee the well-formedness of (usk, attrs, upk) . In the challenge phase, $\mathcal{A}$ chooses two arrays of attributes $\text{attrs}_0, \text{attrs}_1$ such that $|\text{attrs}_0| = |\text{attrs}_1| = l$, and $\forall i \in \text{idx}, \text{attrs}_0[i] = \text{attrs}_1[i]$. $\mathcal{C}$ selects $b \in \{0, 1\}$, and generates ucred_b as the response to $\mathcal{A}$'s oracle query. $\mathcal{A}$ is still allowed to query the oracle $\mathcal{O}_{\text{ObtC}}$. Finally, $\mathcal{A}$ outputs a guess b' . This experiment then outputs whatever $\mathcal{A}$ outputs. By definition, $\Pr[\text{Expt}_A^b(1^\lambda) = 1] = \Pr[E_b]$.

Game G_2 : In this game, the following modification is introduced. Instead of generating the NIZK proof Π_{Sign} faithfully, $\mathcal{C}$ simulates it without using the witness of form Eq. (4). This is achieved by running the ZK simulator of Π_{Sign} . Note that now the challenge registration credential does not depend on the challenger's bit b and is computationally close to the one in **Game G_b** . Hence $|\Pr[E_b] - \Pr[E_2]| \leq \text{negl}[\lambda]$ holds for any $\mathcal{A}$. $\square$

Theorem 4.3. *Our system satisfies the unlinkability of tokens if the NIZK protocols are zero-knowledge.*

Proof. In our system, tok achieves the purpose of designated verifier and token hiding by encrypting $\text{tok}' \leftarrow \text{Prove}_{\text{ISIS}}(\sigma_i)$. In this experiment, we show that even if the adversary obtains tok' , the token remains unlinkable. Let $\mathcal{A}$ be any PPT adversary against our LAASSO scheme with advantage ϵ . We will prove $\epsilon = \text{negl}[\lambda]$ using the ZK property of the underlying NIZK systems. Specifically, we construct indistinguishable games G_b, G_2 such that G_b is the experiment $\text{Expt}_A^{\text{UnlTok}}(1^\lambda)$. Denote E_i as the event that G_i outputs 1 for $i \in \{0, 1, 2\}$.

Game G_b : This game is the original unlinkability of tokens experiment $\text{Expt}_A^{\text{UnlTok}-b}(1^\lambda)$ for $b \in \{0, 1\}$. $\mathcal{C}$ runs **Setup**, **ISKeyGen** and **SPKeyGen** algorithms, then sends (pp, ipk, idx) to $\mathcal{A}$. $\mathcal{A}$ queries oracles $\mathcal{O}_{\text{RegC}}, \mathcal{O}_{\text{Obt}}$ to obtain legal registration certificates and tickets. In the challenge phase, $\mathcal{A}$ chooses tk_0 and tk_1 . $\mathcal{C}$ selects $b \in \{0, 1\}$, and generates tok'_b as the response to $\mathcal{A}$'s oracle query. Finally, $\mathcal{A}$ outputs a guess b' . This experiment then outputs whatever $\mathcal{A}$ outputs. By definition, $\Pr[\text{Expt}_A^b(1^\lambda) = 1] = \Pr[E_b]$.

Game G_2 : In this game, the following modification is introduced. Instead of generating the NIZK proof Π_{ISIS} faithfully, $\mathcal{C}$ simulates it without using the witness of form Eq. (3). This is achieved by running the ZK simulator of Π_{ISIS} . Note that now the challenge token does not depend on the challenger's bit b and is statistically close to the one in **Game G_b** . Hence $|\Pr[E_b] - \Pr[E_2]| \leq \text{negl}[\lambda]$ holds for any $\mathcal{A}$. $\square$

Theorem 4.4. *If the MSIS problem is hard, the NIZK system is multi-proof extractable, and the NTRU-ISIS_f signature scheme is one-more unforgeable, then the client's credentials in our system are one-more unforgeable.*

Proof. Suppose a PPT adversary has forged a credential presentation for an honest client or for an unregistered client in the **ObtTkt** algorithm. In

that case, it has succeeded in the credential-unforgeability experiment. We consider two types of adversaries separately. Type-I: $\mathcal{A}$ has forged a credential $\text{ucred}_{\text{attrs}^*}$, where there exists $\{usk, \text{attrs}_{\text{idx}}, upk, *\} \in \mathcal{L}_{\text{HC}}$ such that $\text{attrs}_{\text{idx}} = \text{attrs}_{\text{idx}^*}$; Type-II: $\mathcal{A}$ has forged a credential $\text{ucred}_{\text{attrs}_{j^*}}$, where for any $\{usk_i, \text{attrs}_i, \text{idx}_i, *\} \in \mathcal{L}_{\text{HC}} \cup \mathcal{L}_{\text{CC}}$, there exists at least one index j in both idx_i and idx_{j^*} such that $\text{attrs}_{\text{idx}_i, j} \neq \text{attrs}_{\text{idx}_{j^*}, j}$. Based on **Lemmas 1** and **2**, we can prove that the theorem holds. $\square$

Lemma 1. *Suppose there exists a type I adversary that has a non-negligible probability ϵ of successfully passing the unforgeability experiment of the certificate. In that case, there exists a simulator S that can solve the MSIS problem with probability ϵ/Q_N , where Q_N is the number of honest clients.*

Proof. Suppose $(\{c_0, c_1\}, c = c_0\mathbf{m} + c_1\mathbf{r})$ is an MSIS sample, where $\|\mathbf{m}, \mathbf{r}\|$ is small. S executes **Setup**, **ISKeyGen** and **SPKeyGen** algorithms and then sends (pp, ipk, idx) to $\mathcal{A}$. Here, we consider a type I adversary, that is, $\mathcal{A}$ attempts to simulate a client with the same disclosure attributes as an honest client $(usk^*, \text{attrs}_{\text{idx}^*}, upk^*, *) \in \mathcal{L}_{\text{HC}}$. S hypothesizes that $upk^* \in \mathcal{L}_{\text{HC}}$, and performs the following operation to respond to $\mathcal{A}$'s oracle query:

- $\mathcal{O}_{\text{RegC}}(upk, \text{attrs})$: if $upk \neq upk^*$, then S executes according to the normal algorithm. Otherwise, S sets up $usk = usk^*$, $\text{attrs} = \text{attrs}^*$ and simulates knowledge proof Π_{ISIS} .
- $\mathcal{O}_{\text{CorC}}(upk)$: if $upk \neq upk^*$, then S executes according to the normal algorithm. Otherwise, it returns $\perp$.
- $\mathcal{O}_{\text{Obt}}(upk, \text{attrs}_{\text{idx}})$: if $upk \neq upk^*$, then S executes according to the normal algorithm. Otherwise, S simulates knowledge proof Π_{Sign} .

If S 's guess of upk^* is correct, then the probability of the guess being correct is $1/Q_N$. In this case, if $\mathcal{A}$ successfully forged the proof of upk^* 's credential, then S can use the knowledge extractor of Π_{Sign} to recover the witness (σ, v) . Therefore, S has solved the MSIS problem with probability ϵ/Q_N . $\square$

Lemma 2. *Suppose there exists a type II adversary that has a non-negligible probability ϵ of successfully passing the credential-unforgeability experiment. In that case, there exists a simulator S that has won the one-more unforgeability experiment of the NTRU-ISIS_f signature with the same probability.*

Proof. S executes the unforgeability experiment for the NTRU-ISIS_f signature and receives the public parameters pp and public key ipk . S executes **Setup**, **ISKeyGen** and **SPKeyGen** algorithms and then sends (pp, ipk, idx) to $\mathcal{A}$. S can make a polynomial number of queries Q_S to the interactive NTRU-ISIS_f signature oracle $\mathcal{O}_{\text{Sign}}$ and perform the following operations to respond to $\mathcal{A}$'s oracle queries. S generates (upk, usk, attrs) corresponding to the attribute set attrs and randomness $\mathbf{r}$ for the client, then submits upk to the signature oracle $\mathcal{O}_{\text{Sign}}$. Later, it receives a signature (σ, x) and sets $\text{ucred} = (\mathbf{r}, \sigma, x)$. Because S possesses the private key isk for issuing credentials on behalf of the IS as well as the private key of honest clients, it is possible to simulate the oracles $\mathcal{O}_{\text{RegC}}, \mathcal{O}_{\text{Obt}}$ without interruption. Therefore, for a type II adversary, $\mathcal{A}$ outputs an effective credential showing $(\text{attrs}_{\text{idx}^*}, \text{ucred}^*, \pi_{\text{ucred}^*})$ at the end of the experiment with probability ϵ . S can use the multi-proof extractability of Π_{ISIS} to extract witnesses $(usk_{j^*}, \text{attrs}_{j^*}, *)$ from $\pi_{\text{ucred}_{j^*}}$ sent by $\mathcal{A}$, and for all $\{usk_i, \text{attrs}_i, \text{idx}_i, *\}_{i \in [Q_S]} \in \mathcal{L}_{\text{HC}} \cup \mathcal{L}_{\text{CC}}$, it also satisfies $usk_i \neq usk_{j^*}$, and there exists at least one index j in both idx_i and idx_{j^*} such that $\text{attrs}_{\text{idx}_i, j} \neq \text{attrs}_{\text{idx}_{j^*}, j}$. Therefore, S successfully forged the signature (σ, x) of c^* corresponding to $(usk^*, \text{attrs}^*, *)$ with probability ϵ . $\square$

Theorem 4.5. *If the NTRU-ISIS_f signature scheme is one-more unforgeable and Π_{ISIS} is multi-proof extractable, then the tokens in our system are unforgeable.*

Proof. S receives pp and ipk from the challenger, and embeds ipk as the public key of the identity server in the simulated system. S then generates the remaining system keys, including the service provider's key pair, and sends pp to $\mathcal{A}$. To answer ticket-issuing queries from $\mathcal{A}$, S generates the corresponding message $(dcid, v)$ and submits $(dcid, v)$ to the signing oracle $\mathcal{O}_{\text{Sign}}$. After receiving the signature σ , S returns the

Table 2
Comparison of security properties.

Scheme	Anonymity	Unlinkability	Token hiding	Fine-grained access control	Designated verifier	Quantum resistance
[3]	×	✓	×	×	×	×
[7]	×	×	×	×	×	×
[5]	×	✓	×	×	×	×
[6]	×	✓	×	×	×	×
[11]	✓	✓	×	×	✓	×
[12]	✓	✓	✓	✓	✓	×
[26]	×	×	✓	×	—	✓
LAASSO	✓	✓	✓	✓	✓	✓

✓: Supported; ×: Unsupported; —: Not reported or not applicable.

ticket $\text{tk} = (dcid, \sigma, v)$ to $\mathcal{A}$ and records $(dcid, v)$ in the list $\mathcal{L}_{\text{Tok}}$. Suppose that $\mathcal{A}$ outputs a forged token $\text{tok}^* = (\text{tok}_1^*, \text{tok}_2^*)$ that is accepted by the verification algorithm with probability ϵ . Since S generated the service provider's private key, it can decrypt tok^* and recover the hidden ticket $\text{tk}^* = (dcid^*, \sigma^*, v^*)$. Because tok^* is valid, the verification algorithm accepts σ^* as a valid NTRU-ISIS_f signature on $(dcid^*, v^*)$. Moreover, by the freshness condition of the token-forgery experiment, $(dcid^*, v^*) \notin \mathcal{L}_{\text{Tok}}$. Therefore, S outputs $(dcid^*, v^*, \sigma^*)$ as a valid forgery against the NTRU-ISIS_f signature scheme. Hence, if $\mathcal{A}$ forges a valid token with probability ϵ , then S breaks the unforgeability of the underlying NTRU-ISIS_f signature scheme with probability at least ϵ . $\square$

Theorem 4.6. *Our system satisfies token hiding if the NIZK protocol is zero-knowledge and the PKE scheme is CPA-secure.*

Proof. In **SignOn**, we use a zero-knowledge proof to confirm that the client holds a legitimate ticket, and we encrypt the ticket information using AES and encrypt the key K using Kyber. Therefore, based on the definitions of zero-knowledge and CPA security, our system satisfies the token-hiding property. $\square$

5. Performance evaluation

This section presents a comprehensive analysis of the performance of LAASSO, including security features, computational cost, and storage overhead.

5.1. Theoretical analysis

We compare the security features of LAASSO with related works [3, 5–7, 11, 12, 26] in Table 2.

- **Anonymity.** This property means that the client remains anonymous to the IS, and the IS cannot obtain private information, such as the attributes of the registered client.
- **Unlinkability.** This property means that the client is anonymous to the SP. The SP is unable to associate the services requested by the client each time.
- **Token hiding.** This property refers to the fact that non-designated verifiers cannot distinguish between the tokens of two different tickets.
- **Fine-grained access control.** This property refers to fine-grained access control based on attributes. When the client obtains the tickets from the ticket provider (i.e., the IS) and accesses the SP, it only needs to disclose the minimum set of attributes, rather than all of them.
- **Designated verifier.** This property means that only the designated SP can decrypt and verify the client's token. This prevents tickets from being misused by a curious SP after they have been obtained.
- **Quantum resistance.** This property refers to the ability to resist quantum computing attacks.

Here, [3, 5–7, 11, 12] are ASSO systems constructed based on the hardness of classical problems, such as bilinear pairing problems, and therefore do not resist quantum attacks. Although [26] is an SSO system based on lattice construction, it lacks anonymity. Note that only the LAASSO system supports all security properties.

Table 3
Theoretical computation overhead.

Algorithms	LAASSO
Setup	$O(dtS_{int})$
ISKeyGen	$O(S_{Tp}) + O(dS_{int})$
SPKeyGen	$O(dS_{int}) + O(dS_{cbd})$
CKeyGen	$O((T_h) + O(dS_u) + O(d \log dT_{int}))$
CReg.C	$O(m_1 \hat{d} S_{gs}) + O(m_1^2 \hat{d} \log dT_{int}) + O(T_h) + O(m_1 \hat{d} S_{rej})$
CReg.IS	$O(m_1^2 \hat{d} \log dT_{int}) + O(T_h) + O(S_{pre})$
ObtTkt.C	$O((d' \log d' + m_1^2 \hat{d} \log dT_{int}) + O(m_1 \hat{d} S_{gs}) + O(T_h) + O(m_1 \hat{d} S_{rej}))$
ObtTkt.IS	$O(m_1^2 \hat{d} \log dT_{int}) + O(T_h) + O(T_h) + O(S_{pre})$
SignOn	$O(m_1 \hat{d} S_{gs}) + O(m_1^2 \hat{d} \log dT_{int}) + O(T_h) + O(m_1 \hat{d} S_{rej}) + O(S_{cbd})$
Verify	$O(m_1^2 \hat{d} \log dT_{int}) + O(T_h)$

Table 4
Notations for basic operations.

Notation	Description
T_h	Mapping to $[-\psi, \psi]^h$ hash operation
T_h	Hash operation in NIZKPoK
T_{int}	Integer multiplication on $\mathbb{Z}_q$
S_{int}	Uniform sampling over $\mathbb{Z}_q$
S_q	Uniform sampling over $[-\psi, \psi]$
S_{cbd}	The central binomial distribution sampling with parameter ψ
S_{Tp}	NTRU trapdoor sampling
S_{pre}	Gaussian preimage sampling
S_{gs}	Gaussian sampling
S_{rej}	Rejection sampling

We mainly consider time-consuming operations, such as NTRU trapdoor generation, hash operation, sampling, integer multiplication on $\mathbb{Z}_q$, as shown in Table 4. We show the computational costs of the algorithms in the LAASSO system in Table 3. Here, d and $\hat{d}$ represent the polynomial degrees used in the signature scheme and the NIZKPoK, respectively, while m_1 is a parameter related to the crs in the NIZKPoK.

To the best of our knowledge, few existing works provide a concrete post-quantum ASSO construction with these combined properties. We compared our work with an existing post-quantum single sign-on system based on lattice problems. Because ISs need to manage a large number of clients, we mainly compare the storage costs of the registration information and registration credentials with [26], as shown in Table 5. Here, the registration information denotes the client's public key in our system and the client's secret share in [26], respectively. Let $d = O(n)$, $I = O(1)$, and $k = O(I^{1+\sqrt{2}}n^{1.5})$, where n denotes the lattice-dimension parameter and I denotes the number of ISs. The registration information in [26] scales as $\Omega(I^{1+\sqrt{2}}n^{1.5})$, whereas in LAASSO it scales as $\Omega(In^{1.5})$. Fig. 3 further visualizes these storage costs. With a fixed number of ISs, we observe that our scheme requires less storage for registration information, while the storage required for registration credentials is slightly larger than in [26]. This difference can be attributed to the fact that our credential is represented as a signature, whereas in [26] it is encoded as a hash value. Moreover, Fig. 4 shows that, for the same lattice dimension $n = 128$, our scheme maintains a smaller signature size as the number of clients grows. Although [26] prevents IS collusion by introducing a

Table 5
Comparison of storage costs for lattice-based SSO systems.

Scheme	Registration Information	Registration Credentials
LPBT-SSO[26]	$O(kn \log p)$	$O(n)$
LAASSO	$O(d \log q)$	$O(d)$

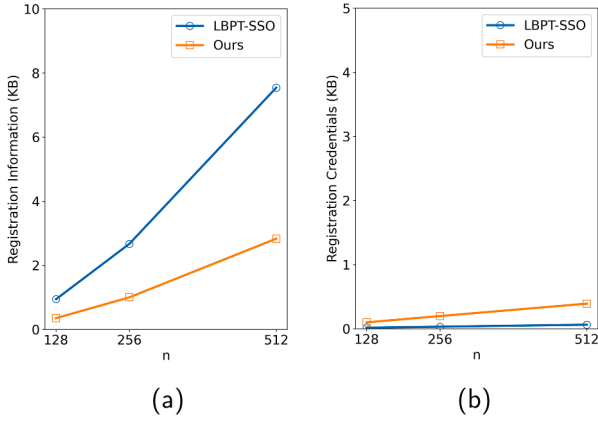


Fig. 3. Comparison of storage costs with $I = 2$ (KB).

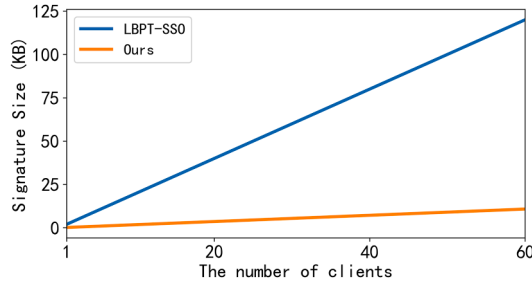


Fig. 4. Comparison of signature sizes with $n = 128$ (KB).

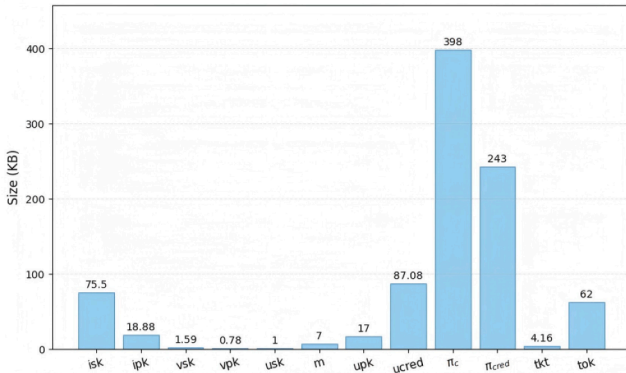


Fig. 5. Component sizes of LAASSO (KB).

threshold secret sharing scheme, it may still expose clients' private information. Furthermore, our solution also supports attribute-based access control, minimizing the exposure of clients' private information.

5.2. Experimental analysis

We present the component sizes of LAASSO at the 128-bit security level in Fig. 5. To achieve this security level, we set the parameters as shown in Table 6. The parameters for the zero-knowledge proof are specified in [19]. The PKE implementation employs Kyber as specified in [30]. In the system, the client signs on by constructing NIZK proofs of possession of registration credentials and tickets.

Table 6
Parameters of LAASSO.

Notations	d	q	t	h	$\mathfrak{s}$	m	ℓ_r	ℓ_m
Value	4096	$\approx 2^{34}$	640	512	$\approx 2^{27}$	3	2	1
Notations	d'	p	t'	k	g'	l	ψ	ℓ_l
Value	1024	$\approx 2^{15}$	256	3072	658	7	2	3

Table 7
Running time of LAASSO (ms).

Algorithm	Party	Mean time	Std. dev.
CReg	C	766.26	142.65
CReg	IS	311.07	54.36
ObtTkt	C	779.70	167.87
ObtTkt	IS	305.04	55.23
SignOn	C	492.67	93.21
Verify	SP	204.85	39.82

Fig. 5 shows that the size of an LAASSO access token is approximately 62 KB. This size is larger than that of classical ASSO constructions based on bilinear pairings [12], mainly because the token contains a lattice-based zero-knowledge proof. This proof is introduced to ensure token unlinkability, and it accounts for most of the token size and the communication overhead in the sign-on phase. From the phase-level perspective, the registration communication cost is incurred only when a user registers with an IS, while in the sign-on phase the main communication comes from transmitting the token. This transmission occurs when establishing an authenticated service session rather than in every subsequent application-layer message. The token is transmitted only once during each sign-on session and therefore mainly represents a one-time bandwidth overhead. Overall, these costs remain reasonable considering the provided post-quantum security and privacy protection.

We implemented LAASSO using open-source libraries: LaZer,³ OpenSSL,⁴ and SageMath.⁵ The experiments were conducted on a Windows 11 device with an Intel Core Ultra 7 255HX processor. Each experiment was repeated 100 times, and the mean running time and standard deviation (Std. dev.) are reported in Table 7. As shown in Table 7, the client-side operations take more time than the IS- and SP-side operations. The registration phase takes about 1.08 s in total for the client and the IS, but this cost is incurred only when a user registers with an IS. For an authentication session after registration, the service sign-on procedure consisting of SignOn and Verify takes about 0.70 s. Compared with the classical pairing-based SSO system PriSign [12], whose reported running times are 13.2 ms/6.2 ms for client/IS registration, 55.5 ms/55.2 ms for client/IS ticket obtaining, 37.6 ms for sign-on, and 36.6 ms for verification, LAASSO requires longer execution time. The additional overhead mainly comes from the lattice-based zero-knowledge proof and post-quantum cryptographic components used to provide quantum resistance, token hiding, and selective attribute disclosure. In particular, proof generation involves rejection sampling and may require several retries. Our implementation without dedicated optimization provides a conservative estimate of the performance. These results indicate that LAASSO is suitable for server-assisted banking SSO on general-purpose client devices, such as browser-based online banking and mobile banking authentication, while smart-card deployment may benefit from platform-specific optimization.

6. Conclusion

In this paper, we propose LAASSO, a lattice-based anonymous single sign-on system that supports attribute-based access policies and of-

³ <https://github.com/lazer-crypto/lazer>

⁴ <https://openssl-library.org/>

⁵ <https://www.sagemath.org/>

fers post-quantum resistance. Clients can anonymously register with the identity server. Here, anonymity refers to the non-disclosure of sensitive personal attributes and private keys. To request services, legally registered clients obtain policy-compliant tickets from the identity server by selectively revealing attributes that satisfy the access policy. These tickets are then presented to the designated service provider in an unlinkable manner, ensuring that ticket contents are not disclosed to unauthorized or non-designated service providers. We also achieve token hiding to prevent curious service providers from learning ticket information. All cryptographic primitives in our scheme are lattice-based and inherently resist quantum adversaries. Replay and double-spending attacks are mitigated by using fresh ticket identifiers, validity periods, and redemption checks. Finally, we evaluate the computational, storage, and communication performance of LAASSO.

CRedit authorship contribution statement

Shunqi Ma: Writing – original draft, Formal analysis; **Rui Shi:** Writing – review & editing, Methodology, Investigation; **Huiyan Chen:** Writing – review & editing, Software, Methodology; **Xi Lin:** Writing – original draft, Investigation, Formal analysis, Conceptualization; **Huaxiong Wang:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the BUCT Excellent Ph.D. Students Foundation (Grant Number: CX20241058), the [Fundamental Research Funds for the Central Universities](#) (Grant Nos. 3282024048, 3282024019 and 3282025004), and the [National Natural Science Foundation of China](#) (Grant No. 62562024). We thank Khai Hanh Tang from [Nanyang Technological University](#) for the assistance provided in this work.

Data availability

Data will be made available on request.

References

- [1] Recordon D, Reed D. OpenID 2.0: a platform for user-centric identity management. In: Proceedings of the second ACM workshop on digital identity management. 2006, p. 11–16.
- [2] Kerberos M. Kerberos: the network authentication protocol. 2017. <https://web.mit.edu/kerberos/>.
- [3] Elmufiti K, Weerasinghe D, Rajarajan M, Rakocevic V. Anonymous authentication for mobile single sign-on to protect user privacy. *Int J Mob Commun* 2008;6(6):760–9.
- [4] Kirkman RA. Evaluating single point failures for safety & reliability. *IEEE Trans Reliab* 2009;28(3):259–63.
- [5] Wang J, Wang G, Susilo W. Anonymous single sign-on schemes transformed from group signatures. In: 2013 5th international conference on intelligent networking and collaborative systems. IEEE; 2013, p. 560–7.
- [6] Asghar MR, Backes M, Simeonovski M. Prima: privacy-preserving identity and access management at internet-scale. In: 2018 IEEE International conference on communications (ICC). IEEE; 2018, p. 1–6.
- [7] Han J, Mu Y, Susilo W, Yan J. A generic construction of dynamic single sign-on with strong security. In: International conference on security and privacy in communication systems. Springer; 2010, p. 181–98.
- [8] Han J, Chen L, Schneider S, Treharne H, Wesemeyer S. Anonymous single-sign-on for n designated services with traceability. In: European symposium on research in computer security. Springer; 2018, p. 470–90.
- [9] Au MH, Susilo W, Mu Y. Constant-size dynamic k-TAA. In: De Prisco R, Yung M, (Eds.), Security and cryptography for networks. Springer: Berlin, Heidelberg; 2006, p. 111–25.
- [10] Camenisch J, Kiayias A, Yung M. On the portability of generalized schnorr proofs. In: Annual international conference on the theory and applications of cryptographic techniques. Springer; 2009, p. 425–42.
- [11] Han J, Chen L, Schneider S, Treharne H, Wesemeyer S, Wilson N. Anonymous single sign-on with proxy re-verification. *IEEE Trans Inf Forensics Secur* 2020;15: 223–36.
- [12] Shi R, Yang Y, Xie H, Feng H, Shi G, Zhang J. Prisign, a privacy-preserving single sign-on system for cloud environments. *Appl Sci* 2023;13(2):727.
- [13] Han J, Chen L, Schneider S, Treharne H, Wesemeyer S. Privacy-preserving electronic ticket scheme with attribute-based credentials. *IEEE Trans Dependable Secure Comput* 2019;18(4):1836–49.
- [14] Feng H, Shi R, Yuan F, Li Y, Yang Y. Efficient strong privacy protection and transferable attribute-based ticket scheme. *J Commun* 2022;43(3):63–75.
- [15] Shi R, Feng H, Xie H, Shi G, Liu B, Yang Y. Privacy-preserving attribute ticket scheme based on mobile terminal with smart card. *J Commun* 2024;43(10):26–41.
- [16] Camenisch J, Neven G, Rückert M. Fully anonymous attribute tokens from lattices. In: Visconti I, De Prisco R, editors. Security and cryptography for networks. Springer: Berlin, Heidelberg; 2012, p. 57–75.
- [17] Nguyen PQ, Zhang J, Zhang Z. Simpler efficient group signatures from lattices. In: Public-key cryptography – PKC 2015. Springer: Berlin, Heidelberg; 2015, p. 401–26.
- [18] Lyubashevsky V, Nguyen NK, Plançon M, Seiler G, Wang H. Shorter lattice-based group signatures via “almost free” encryption and other optimizations. In: Advances in cryptography – ASIACRYPT 2021. Springer International Publishing; 2021, p. 218–48.
- [19] Bootle J, Lyubashevsky V, Nguyen NK, Sorniotti A. A framework for practical anonymous credentials from lattices. In: Annual international cryptology conference. Springer; 2023, p. 384–417.
- [20] Lai Q, Liu F-H, Lysyanskaya A, Wang Z. Lattice-based commit-transferrable signatures and applications to anonymous credentials. *IACR Cryptol ePrint Arch* 2023;2023:766.
- [21] Jeudy C, Roux-Langlois A, Sanders O. Lattice signature with efficient protocols, application to anonymous credentials. In: Handschuh H, Lysyanskaya A, editors. Advances in cryptography – CRYPTO 2023. Springer Nature: Switzerland, Cham; 2023, p. 351–83.
- [22] Bootle J, Lyubashevsky V, Seiler G. Algebraic techniques for short(er) exact lattice-based zero-knowledge proofs. In: Boldyreva A, Micciancio D, editors. Advances in cryptography – CRYPTO 2019. Cham: Springer International Publishing; 2019, p. 176–202.
- [23] Eskin MF, Steinfeld R, Liu JK, Liu D. Lattice-based zero-knowledge proofs: new techniques for shorter and faster constructions and applications. In: Boldyreva A, Micciancio D, editors. Advances in cryptography – CRYPTO 2019. Cham: Springer International Publishing; 2019, p. 115–46.
- [24] Lyubashevsky V, Nguyen NK, Plançon M. Lattice-based zero-knowledge proofs and applications: shorter, simpler, and more general. In: Annual international cryptology conference. Springer; 2022, p. 71–101.
- [25] Argo S, Güneysu T, Jeudy C, Land G, Roux-Langlois A, Sanders O. Practical post-quantum signatures for privacy. In: Proceedings of the 2024 on ACM SIGSAC conference on computer and communications security. 2024, p. 1523–37.
- [26] Cao C, Xu C, Jiang C, Zhang Z, Dong X, Chen K. LPBT-SSO: password-based threshold single-sign-on authentication from LWE. *IEEE Trans Dependable Secure Comput* 2024;22(1):491–504.
- [27] Ducas L, Lyubashevsky V, Prest T. Efficient identity-based encryption over NTRU lattices. In: Sarkar P, Iwata T, editors. Advances in cryptography – ASIACRYPT 2014. Springer: Berlin, Heidelberg; 2014, p. 22–41.
- [28] Prest T, Fouque P-A, Hoffstein J, Kirchner P, Lyubashevsky V, Pornin T, Ricosset T, Seiler G, Whyte W, Zhang Z, et al. Falcon: Fast-Fourier lattice-based compact signatures over NTRU. Submission to the NIST's Post-Quantum Cryptography Standardization Process 2018;36(5):1–75.
- [29] del Pino R, Katsumata S. A new framework for more efficient round-optimal lattice-based (partially) blind signature via trapdoor sampling. In: Annual international cryptology conference. Springer; 2022, p. 306–36.
- [30] Avanzi R, Bos J, Ducas L, Kiltz E, Lepoint T, Lyubashevsky V, et al. Crystals-kyber algorithm specifications and supporting documentation. NIST PQC Round 2019;2(4):1–43.
- [31] Standards N. Advanced encryption standard (AES). 2001. <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.197.pdf>.
- [32] Ducas L, Nguyen PQ. Learning a zonotope and more: cryptanalysis of NTRUSign countermeasures. In: International conference on the theory and application of cryptography and information security. Springer; 2012, p. 433–50.